



科学研究的人工智能方法学

复旦大学人工智能创新与产业研究院研究员
徐盈辉

未来25年，这125个问题将让科学家“大伤脑筋”

这125个问题，其中涉及生命科学的问题占46%，关系宇宙和地球的问题占16%，与物质科学相关的问题占14%以上，认知科学问题占9%。其余问题分别涉及数学与计算机科学、政治与经济、能源、环境和人口等。

一场正在发生的科研范式变革



《 Science 》刊登麻省理工
AI药物逆合成设计论文，AI合
成了**15个**药物分子

2019.3

利物浦大学使用AI发现**4种**可
传导锂的新固态材料，可提
供更长续航能力

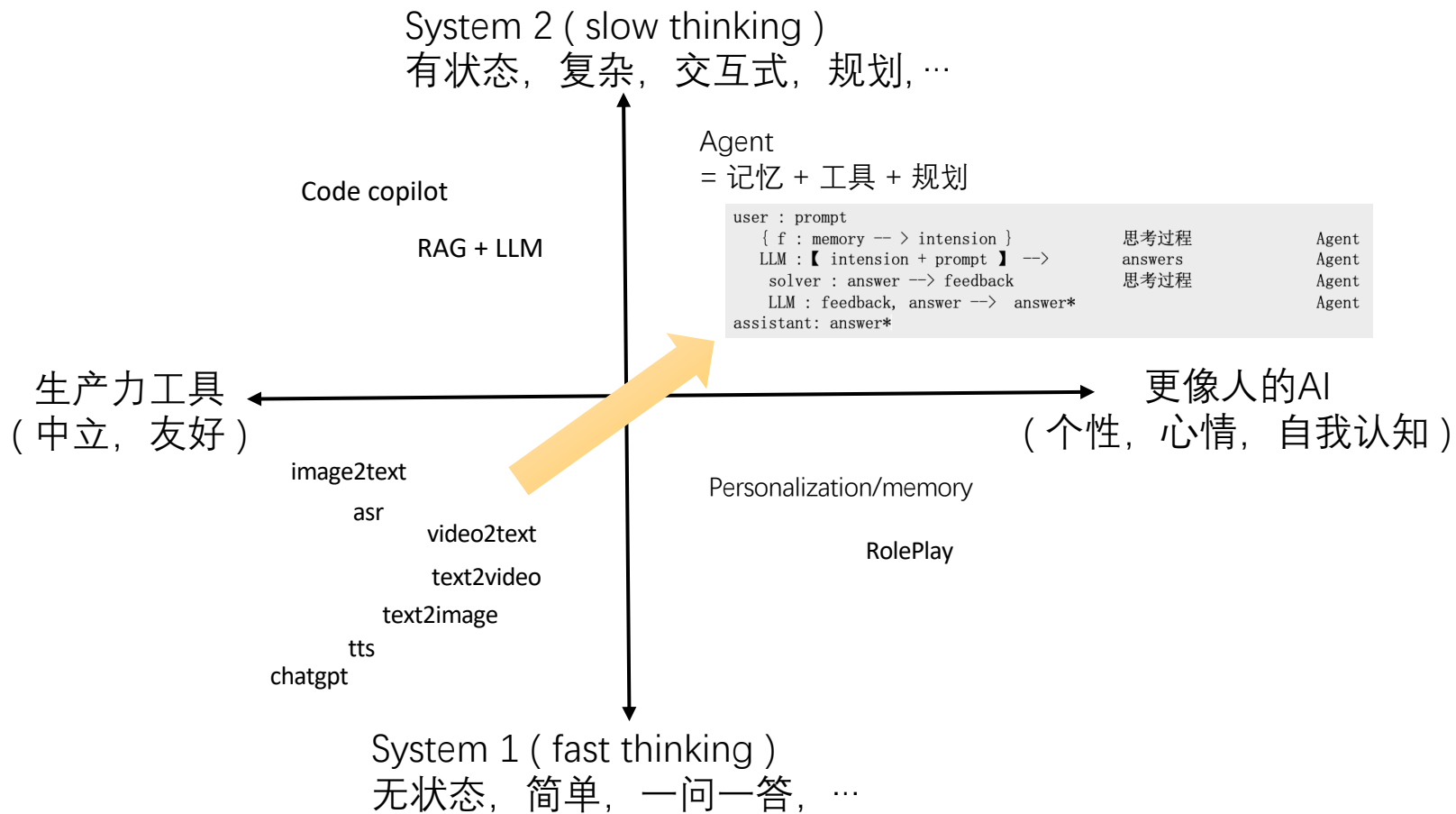
2021.5

Deepmind宣布开放包括多
个动物、植物的**2亿**个蛋白质
结构的预测结果

2022.7



大模型时代(快与慢)



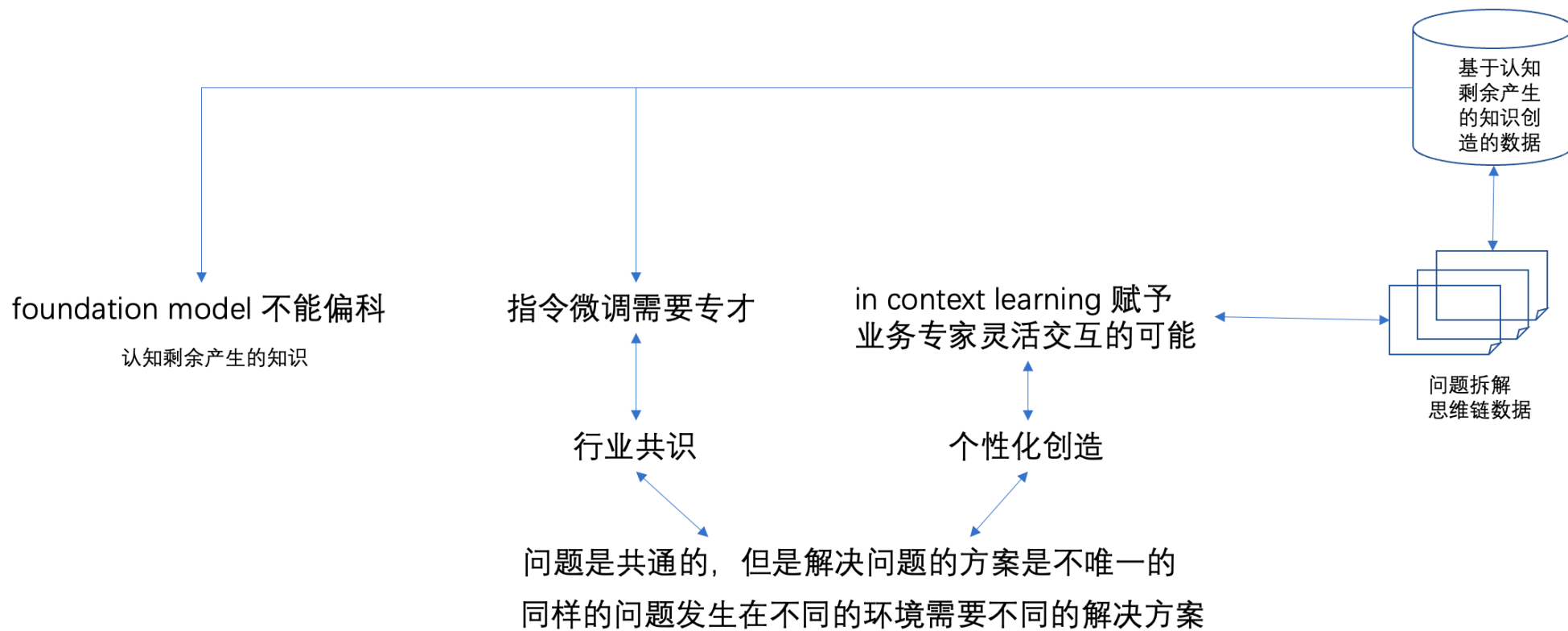
丹尼尔·卡尼曼



丹尼尔·卡尼曼



大模型的基础工作范式



挑战1 – 语料工程

行业高质量语料的供给，需要面对的不仅仅是一维线性的文字，还需要面对表格，图像，语音，图像，视频等；

很多专业的行业语料是高维且高阶的信息源；数据准备很明显是专业的人在干专业的事情，对于信息的二次加工和专业知识的提炼，需要专业人士自己来砸自己的饭碗。

合成数据

- 人工智能生成
- 物理仿真引擎生成
- 求解器生成
- 专家规则引擎/知识图谱

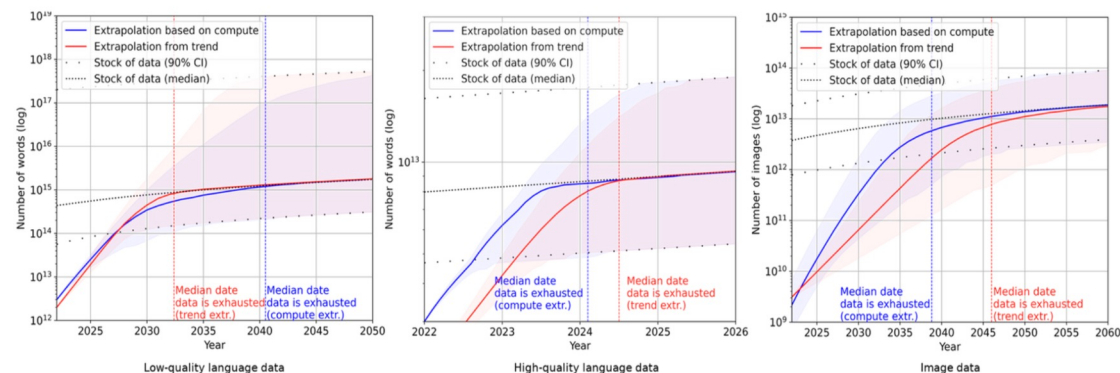


Figure 1: ML data consumption and data production trends for low quality text, high quality text and images.

Ref: stateof.ai 2023



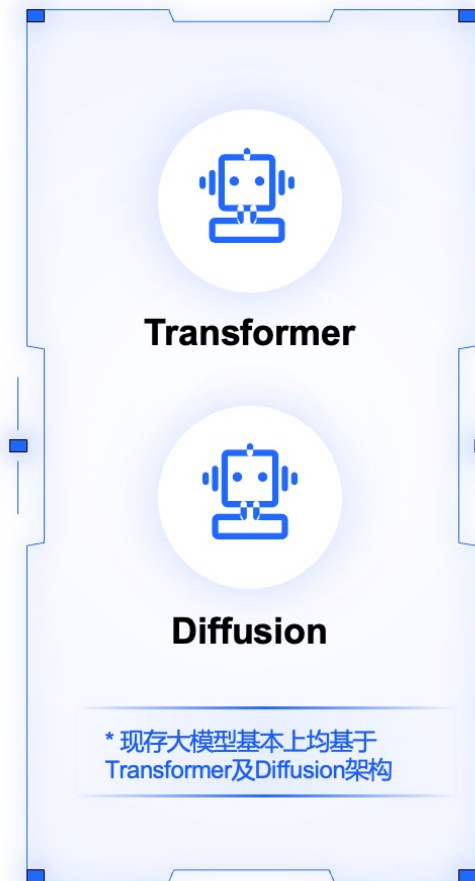
挑战2 - 幻觉

“

In some sense, hallucination is all LLMs do. They are dream machines.

— Andrej Karpathy, OpenAI co-founder

”



神经网络架构会导致幻觉问题
目前生成内容不可控制

- 过拟合和数据不足导致网络学习到噪声而非潜在数据分布
- 模型复杂度过高而缺乏有效的正则化措施

数据源

- 捕获事实知识利用率低
- 长尾知识回忆不足

训练过程

- 架构缺陷、曝露偏差
- 存在能力错位、信念错位

推理

- 固有的抽样随机性
- 不完美的解码表示

事实性幻觉

忠实性幻觉



医疗保健：误诊及错判
金融法律：指令错误
社会领域：虚假信息传播



生成导致有害后果的虚假信息的责任方判定



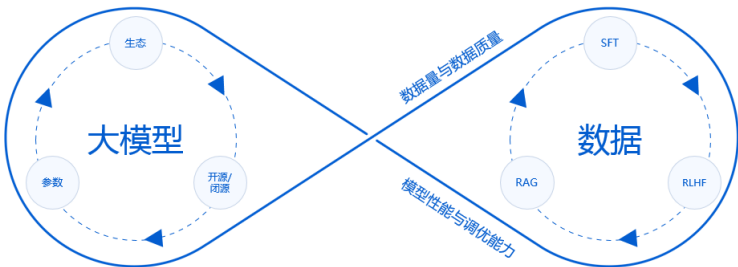
幻觉问题会使得人们对AI的可靠度和可信度产生质疑，对系统的不信任会影响对其应用及发展进步

注：事实性幻觉是指模型生成的内容与可验证的现实世界事实不一致，忠实性幻觉是指模型生成的内容与用户的指令或上下文不一致

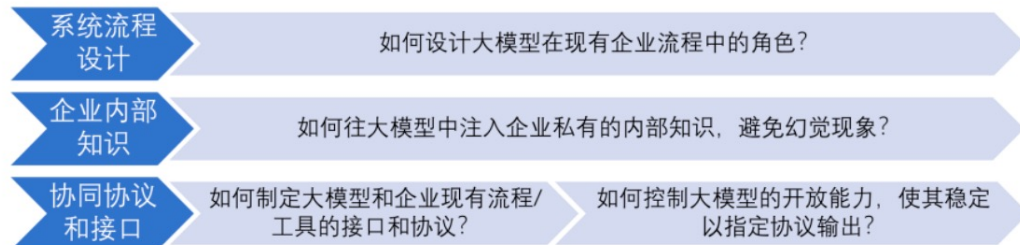


挑战3 – 领域适配问题

全量数据与大模型场景



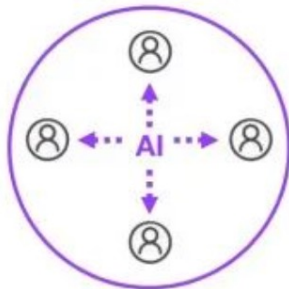
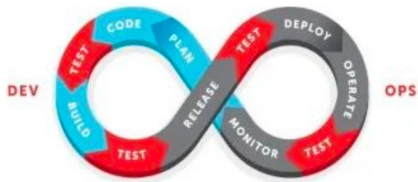
在 toB 应用中需要的是协同和可控



1968年, NATO会议, 软件工程学科诞生

2001年, 敏捷宣言发布

2023年, 大模型爆发式产生



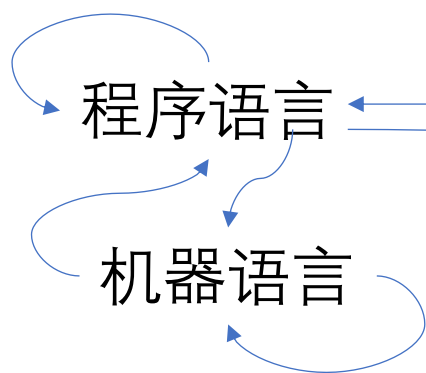
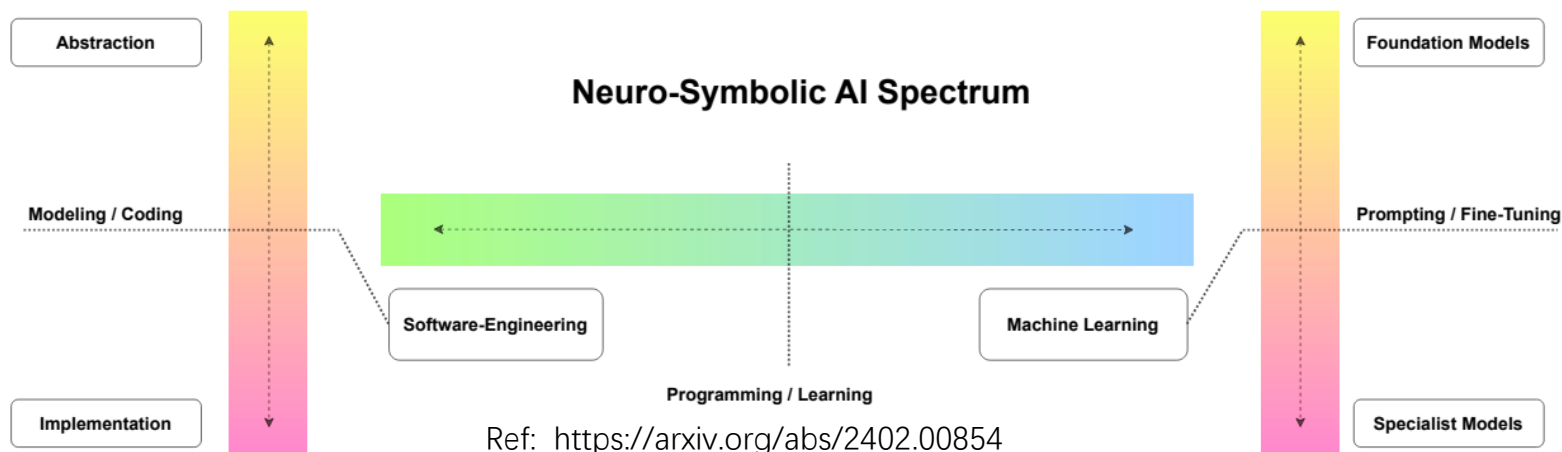
代码智能化、极简开发、极致体验

在既存的业务运转流程中实现大模型应用的深入落地, 面临的是:

1. 软件1.0/2.0模式下的数据流转的并非大模型友好
2. 大模型需要时间与现有流程的融合和适配。
3. 大模型领域适配的数据制备需要领域专家的参与



挑战3 – 逻辑推理&复杂决策



	程序语言	基础语言
逻辑性	强	弱
符号性	强	弱
结构性	强	弱
严谨性	强	弱
容错性	弱	强

基础语言

1. 训练数据的局限性
2. 缺乏结构化知识
3. 有限的上下文理解
4. 缺乏元认知能力（反思/改进）

大模型在复杂决策场景仍有缺陷。尽管大模型在聊天机器人等开放式闲聊场景中表现出色，但在金融、医疗等严肃场景中，更关注的是corner case带来的企业风险。从开放式闲聊到复杂决策，大模型还有很长的路要走。



挑战3 – 大模型的格式化输出问题

Low-level约束类型:

1. **格式约束**: 控制输出的具体格式, 如表格、列表、JSON等。
2. **长度约束**: 限制生成文本的字数或段落数量。

High-level约束类型:

1. **语义约束**: 确保输出符合特定的语义要求或主题。
2. **风格约束**: 控制输出的写作风格, 如正式、非正式、技术等。

多样性和复杂性:

用户需求多样, 结构化输出需要满足各种不同的格式和语义要求。

上下文依赖:

生成内容需根据特定上下文精确调整, 这增加了复杂性。

模型局限性:

当前LLM在严格遵守结构化约束方面能力有限。



格式化输出意义: 约束输出可以提高开发效率、与工作流程的集成、用户体验和信任度





物理视角的大模型

Generative modeling



Known: samples

Unknown: generating distribution

Learning from data

$$\mathcal{L} = -\mathbb{E}_{x \sim \text{data}} [\ln p(x)]$$

Statistical physics



Known: energy function

Unknown: samples, partition function

Learning from energy

$$F = \mathbb{E}_{x \sim p(x)} [E(x)] + k_B T \mathbb{E}_{x \sim p(x)} [\ln p(x)]$$

$$p(x) = \frac{e^{-E(x)/k_B T}}{Z}$$

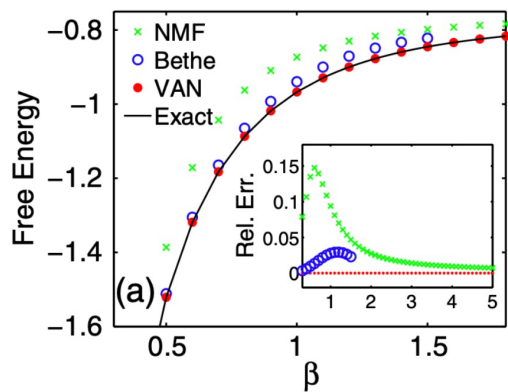
$$\mathcal{L} = \mathbb{E}_{X \sim p_{\text{data}}} [\ln p_{\theta}(X)] = \sum_i \mathbb{E}_{X \sim p_{\text{data}}} [\ln p_{\theta}(x_i | x_{<i})]$$

Variational autoregressive network for statistical mechanics

$$p(X) = \prod_i p(x_i | x_{<i})$$



Sherrington-Kirkpatrick spin glass



Naive mean-field
factorized probability

$$p(X) = \prod_i p(x_i)$$

Bethe approximation
pairwise interaction

$$p(X) = \prod_i p(x_i) \prod_{(i,j) \in E} \frac{p(x_i, x_j)}{p(x_i)p(x_j)}$$

Variational autoregressive
network

$$p(X) = \prod_i p(x_i | x_{<i})$$

大语言模型:

- 在文本生成任务中, x_i 表示文本序列中的第 i 个 token, $x_{<i}$ 是该 token 之前的上下文。
- 模型通过预测下一个 token 的条件概率 $p(x_i | x_{<i})$, 逐步生成完整的序列。



自由能最小化原则

$$F = E - TS$$

energy



entropy



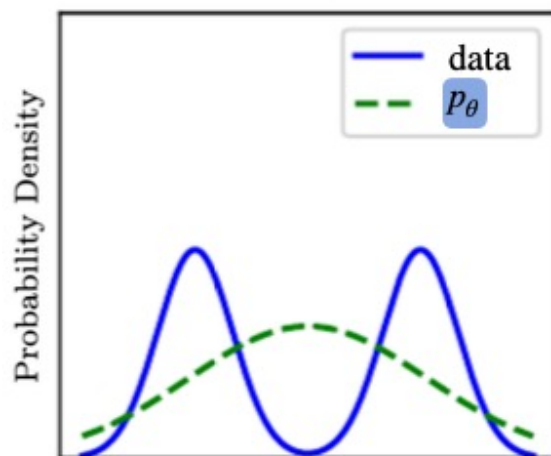
F is a **cost function** of Nature

最小化 F 的过程可以看作在降低“系统的不稳定性”和“对抗外界约束”之间寻找平衡

Forward KL or reverse KL

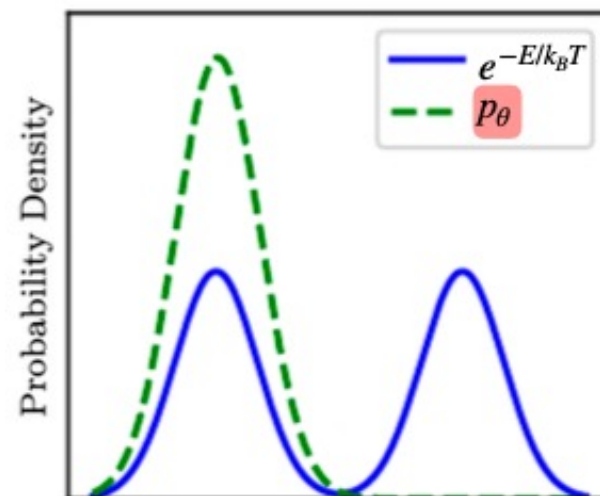
$$\min_{\theta} \text{KL}(p_{\text{data}} \parallel p_{\theta})$$

Mode covering



$$\min_{\theta} \text{KL}(p_{\theta} \parallel e^{-E/k_B T})$$

Mode seeking



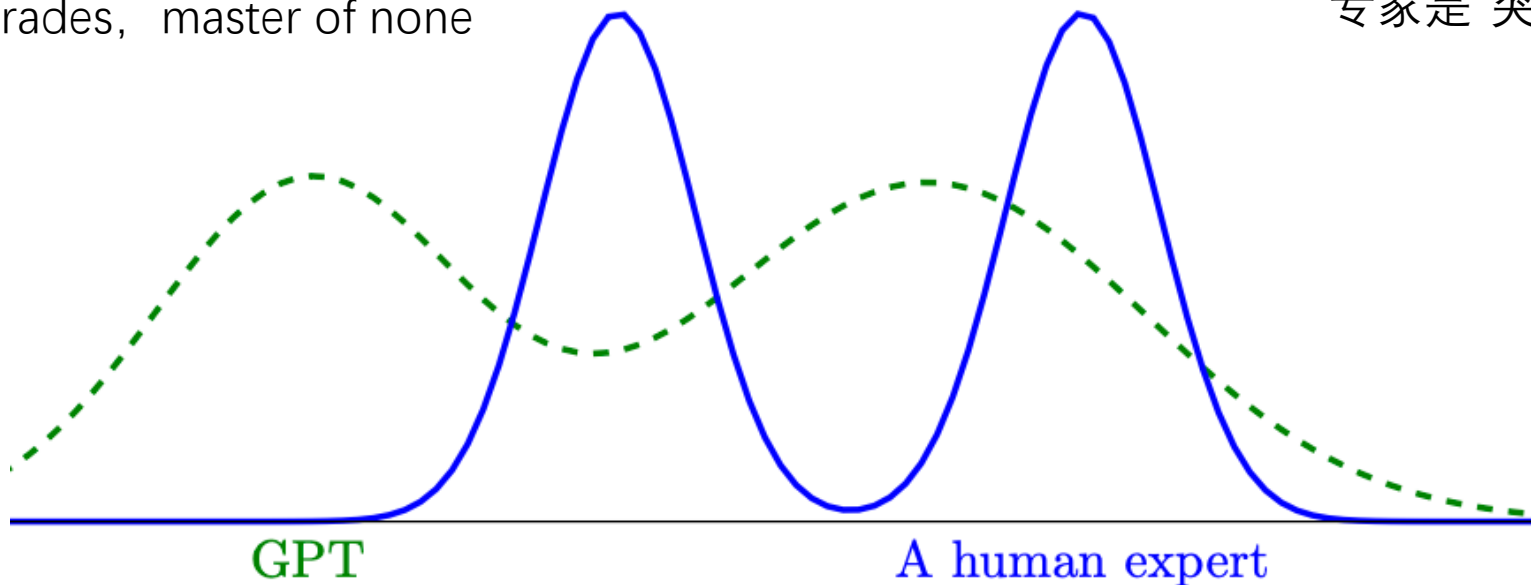


GPT vs. 人类专家

Jack of all trades, master of none

博而不精

GPT 是“填补空白”，力求覆盖完整的分布；
专家是“突破边界”，关注核心模式。



模型追求覆盖所有模式，生成分布更广泛，
适合大规模通用任务。缺点是对低概率区域
可能过拟合，缺乏在特定领域的精确性。

更注重聚焦高概率区域，忽略尾部样本，
适合特定领域高精度任务。缺点是分布覆
盖范围窄，泛化能力较差。



GPT的应用

GPT 的局限性：泛化 vs 精确性

泛化性： GPT 的目标是尽量覆盖所有语言模式，因此它可以在各种任务上表现出一定的能力。

精确性： 由于 Forward KL 更关注模式覆盖而非模式集中，GPT 在特定任务或高精度场景（如医学、法律）可能无法与专家相媲美。

优化 GPT 的潜力

为了在特定任务中提升 GPT 的性能，可以引入类似 Reverse KL 的约束，专注于特定领域内的高概率分布模式。例如：

- **微调：** 在特定领域对 GPT 进行额外训练，强化高概率区域的拟合。
- **奖励建模（如 RLHF）：** 加入 KL 散度约束，使生成分布聚焦于人类偏好的高概率模式。

Forward KL 与 Reverse KL 的结合

组合使用：

- Forward KL 强调模式覆盖，适合构建大规模通用模型。
- Reverse KL 强调模式集中，适合在特定任务或场景下提升模型性能。

在实际应用中，可以通过结合这两种优化目标，让模型既具备泛化能力，又能在特定任务中表现出高精度。

Next-word prediction is
massively multi-task learning.

大语言模型的ICL能力



	例子
语法 (Grammar)	In my free time, I like to {run, banana}
词汇语义 (Lexical Semantics)	I went to the zoo to see giraffes, lions, and {zebras, spoon}
世界知识 (World Knowledge)	The capital of Denmark is {Copenhagen, London}
情感分析 (Sentiment Analysis)	Movie review: I was engaged and on the edge of my seat the whole time. The movie was {good, bad}
更复杂的情感分析 (Harder Sentiment Analysis)	Movie review: Overall, the value I got from the two hours watching it was the sum total of the popcorn and the drink. The movie was {bad, good}
翻译 (Translation)	The word for “pretty” in Spanish is {bonita, hola}
空间推理 (Spatial Reasoning)	[...] Iroh went into the kitchen to make some tea. Standing next to Iroh, Zuko pondered his destiny. Zuko left the {kitchen, store}
数学问题 (Math Question)	First grade arithmetic exam: $3 + 8 + 4 = \{15, 11\}$

Extreme multi-task learning!



ICL的核心特性

ICL 的核心特性

1. 无需额外训练:

- 传统模型需要通过显式训练来学习任务，而大语言模型只需要通过构造合适的输入提示 (prompt) 即可完成任务。

2. 基于提示的推理:

- 用户在输入中提供任务描述或少量示例 (如问题-答案对)，模型会根据这些提示调整其响应方式。

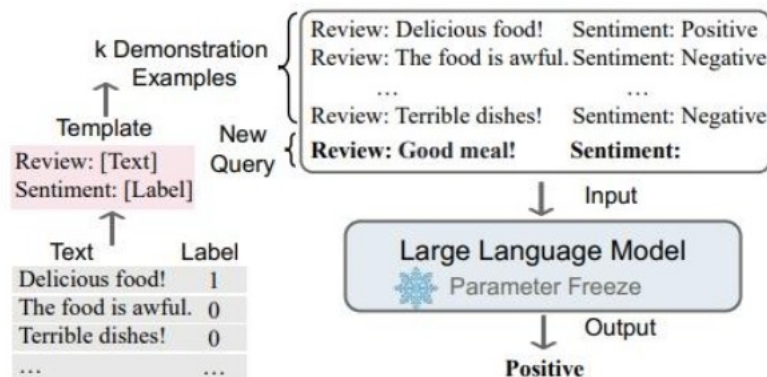
3. 任务灵活性:

- LLM 可以在同一模型中执行多种任务，如分类、翻译、总结、问答等，只需更改输入格式或上下文内容。

4. 零样本与少样本学习:

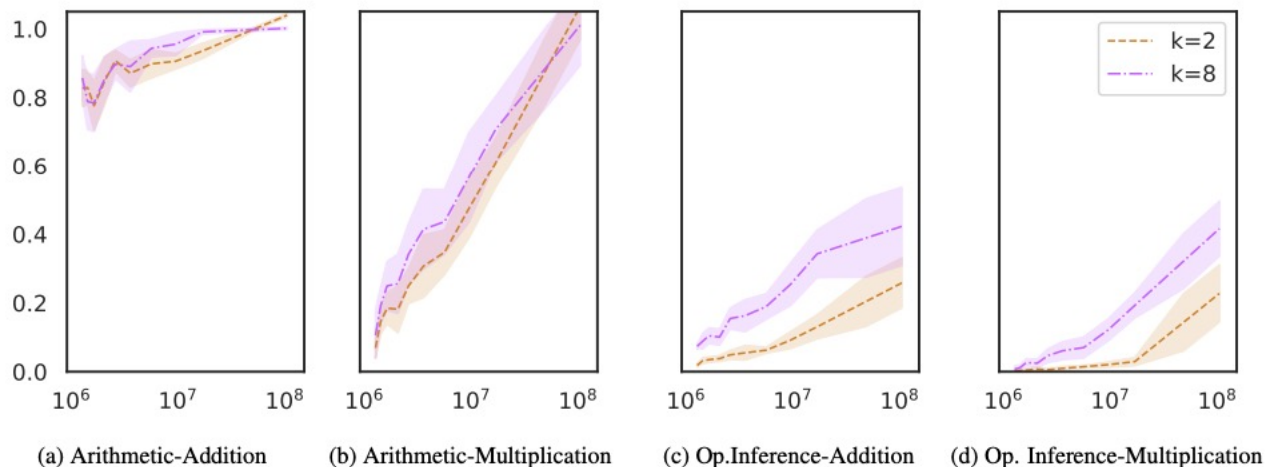
- 零样本学习 (Zero-Shot Learning):** 模型无需任何示例，通过理解提示即可完成任务。
- 少样本学习 (Few-Shot Learning):** 模型通过输入中提供的少量示例进行任务推理。

In-Context Learning (ICL)是指在给Large Language Model (LLM)输入prompt时，在希望做预测的 x_{query} 前面给出一定数量的 (x_i, y_i) 样本对作为demonstration，然后将 $(x_1, y_1), \dots, (x_k, y_k), x_{query}$ 作为整体 (一般样本对之间用"\n"或其他placeholder连接) 输入给模型，目的是希望在不微调模型参数的条件下，让模型从demonstration中“学习”下游任务的知识，从而增强对下游任务的zero-shot预测能力。下面的图1概括了ICL的过程。



Impact of Pretraining Term Frequencies on Few-Shot Reasoning

Yasaman Razeghi¹ Robert L. Logan IV¹ Matt Gardner² Sameer Singh^{1,3}



术语频率 (x轴) 和 in-context learning 性能 (y轴) 之间的相关性; 从左到右: 加法、乘法、提示中无任务指示的加法和提示中无任务指示的乘法; 来自 Razeghi 等人的数据

in-context learning 性能与每个示例中的术语 (数字和单位) 在 GPT-J 的预训练数据 (The PILE) 中出现的次数高度相关

in-context learning 主要是定位在预训练期间学习的潜在概念。特别是, 如果特定实例中的术语在预训练数据中多次出现, 则模型可能会更好地了解输入分布。根据贝叶斯推理, 这将为定位潜在概念以执行下游任务提供更好的信号。



贝叶斯推断的视角：

在统计学中，少样本学习可以看作是一种条件推断：

$$P(y | x, \mathcal{D}) \propto P(\mathcal{D} | x, y)P(y | x)$$

- $\mathcal{D}$ 是 prompt 提供的示例集合。
- $P(\mathcal{D} | x, y)$ 表示示例与任务分布的相容性。
- $P(y | x)$ 是语言模型基于训练数据的先验分布。

通过上下文示例，模型调整先验分布，使得输出更加符合 $\mathcal{D}$ 所描述的规律。

模式识别与推广：

模型在预训练阶段已学会识别输入序列中的模式（如映射关系、句法结构）。prompt 示例强化了这些模式，调整了条件概率分布：

$$P(y | x; \theta, \mathcal{D})$$

其中：

- $\mathcal{D}$ 是 prompt 示例对任务的定义。
- 模型通过最大化上下文的似然函数实现模式推广。

数学直观：Prompt 如何重新分配概率

假设语言模型的输出概率为：

$$P(y | x; \theta) = \prod_{t=1}^T P(y_t | y_{<t}, x; \theta)$$

Prompt 的引入改变了条件上下文 x ，从而动态调整每一步的概率分布：

$$P(y_t | y_{<t}, x, X_{\text{prompt}}; \theta)$$

具体来说，prompt 的示例和描述可以：

- 增强某些词或模式的优先级（提高其概率）。
- 削弱不符合任务描述的生成路径。

通过这种上下文引导，prompt 在固定参数 θ 的前提下，动态地对模型的生成概率分布 $P(y)$ 进行重新分配。

重新分配概率空间的核心逻辑

总结来说，prompt 通过上下文输入改变了：

1. 注意力权重分布：Prompt 改变了 Transformer 对输入序列中各部分的关注程度。
2. 条件上下文表示：Prompt 提供的示例和任务描述改变了模型对生成任务的理解。
3. 概率分布的动态调整：Prompt 的信息对生成的条件概率分布进行“偏置”，在固定参数的情况下重新分配输出空间的权重。



Meta-in-context learning in large language models

Julian Coda-Forno^{1,2,*} Marcel Binz¹ Zeynep Akata²

Matthew Botvinick³ Jane X. Wang³ Eric Schulz¹

¹Max Planck Institute for Biological Cybernetics, ²University of Tübingen - Tübingen, Germany;

³Google DeepMind - London, United-Kingdom

*{julian.coda-forno@tuebingen.mpg.de}

MetaICL: Learning to Learn In Context

Sewon Min^{1,2} Mike Lewis² Luke Zettlemoyer^{1,2} Hannaneh Hajishirzi^{1,3}

¹University of Washington ²Meta AI ³Allen Institute for AI

{sewon,lsz,hannaneh}@cs.washington.edu mikelewis@fb.com

Abstract

Large language models have shown tremendous performance in a variety of tasks. In-context learning – the ability to improve at a task after being provided with a number of demonstrations – is seen as one of the main contributors to their success. In the present paper, we demonstrate that the in-context learning abilities of large language models can be recursively improved via in-context learning itself. We coin this phenomenon *meta-in-context learning*. Looking at two idealized domains, a one-dimensional regression task and a two-armed bandit task, we show that meta-in-context learning adaptively reshapes a large language model’s priors over expected tasks. Furthermore, we find that meta-in-context learning modifies the in-context learning strategies of such models. Finally, we extend our approach to a benchmark of real-world regression problems where we observe competitive performance to traditional learning algorithms. Taken together, our work improves our understanding of in-context learning and paves the way toward adapting large language models to the environment they are applied purely through meta-in-context learning rather than traditional finetuning.

MetaICL: Learning to Learn In Context

Sewon Min^{1,2} Mike Lewis² Luke Zettlemoyer^{1,2} Hannaneh Hajishirzi^{1,3}

¹University of Washington ²Meta AI ³Allen Institute for AI

{sewon, lsz, hannaneh}@cs.washington.edu mikelewis@fb.com

	Meta-training	Inference
Task	C meta-training tasks	An unseen <i>target</i> task
Data given	Training examples $\mathcal{T}_i = \{(x_j^i, y_j^i)\}_{j=1}^{N_i}, \forall i \in [1, C]$ ($N_i \gg k$)	Training examples $(x_1, y_1), \dots, (x_k, y_k)$, Test input x
Objective	For each iteration, 1. Sample task $i \in [1, C]$ 2. Sample $k + 1$ examples from $\mathcal{T}_i$: $(x_1, y_1), \dots, (x_{k+1}, y_{k+1})$ 3. Maximize $P(y_{k+1} x_1, y_1, \dots, x_k, y_k, x_{k+1})$	$\operatorname{argmax}_{c \in C} P(c x_1, y_1, \dots, x_k, y_k, x)$

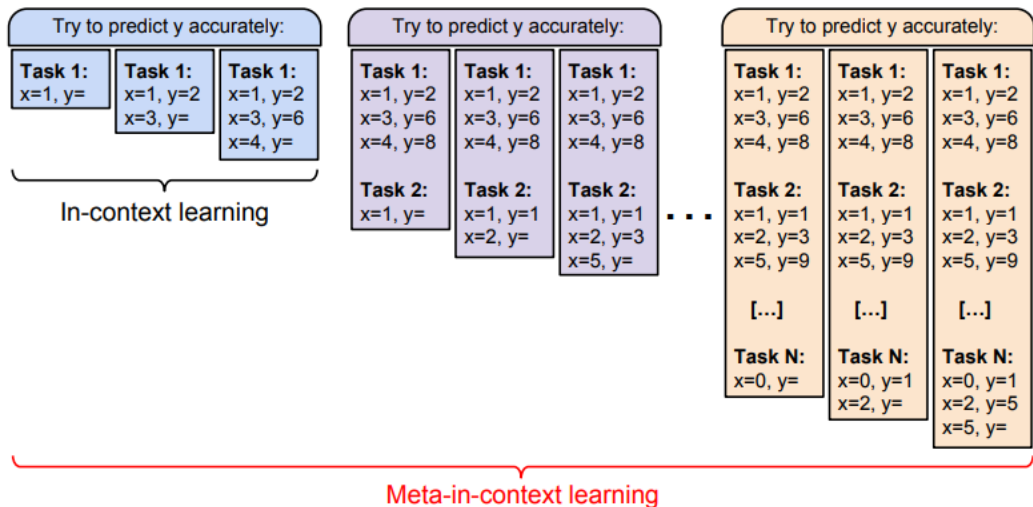
Meta-in-context Learning

Meta-in-context learning 指的是通过上下文学习递归地提升模型的学习能力。换句话说，它利用大语言模型在上下文中学习任务的能力，通过不断暴露于多个任务的上下文示例，使模型不仅能在单个任务中学习，还能在跨任务中适应和改进学习策略。

关键特征：

1. **依赖上下文而非参数调整**：完全基于上下文信息实现学习，无需对模型参数进行传统的微调。
2. **递归增强**：模型通过在多个任务中更新其内部先验分布和学习策略，从而提高对新任务的适应能力。
3. **任务序列驱动**：通过逐步呈现的任务序列，模型能够调整对环境的先验假设，优化跨任务的表现。

实验表明，大语言模型（如GPT-3）通过观察多个任务，能够显著提高新任务中的表现。这一现象被称为“元上下文学习”。



上下文学习 (In-context Learning):

- 左侧蓝色部分展示了单个任务 (Task 1) 的学习过程。
- 模型通过观察任务1的若干输入-输出示例 (如 $x = 1, y = 2, x = 3, y = 6, x = 4, y =$), 尝试预测新输入下的输出值。
- 每个任务的学习是独立的，模型只依赖于该任务上下文中的示例，目标是提高任务1内的预测准确性。

元上下文学习 (Meta-in-context Learning):

- 中间和右侧部分 (紫色和橙色) 展示了模型跨多个任务的学习过程。
- 模型不仅在每个任务 (如Task 1、Task 2.....) 内进行上下文学习，还通过任务间的学习，逐渐调整其对任务分布的**先验假设**和**学习策略**。
- 随着任务序列的增加 (Task 1 到 Task N), 模型在新任务中的初始表现变得更好，体现了元上下文学习的递归改进能力。

递归关系:

- 上下文学习聚焦于单个任务内的改进，而元上下文学习通过多任务序列 (Task 1, Task 2, ... Task N) 的输入，提升模型对新任务的适应能力。

思维链



We posit that chain-of-thought reasoning becomes useful exactly when the training data is structured locally, in the sense that observations tend to occur in overlapping neighborhoods of concepts

- 验证链式思维 (Chain-Of-Verification, CoVe): 强调通过验证每一步的推理来确保答案的准确性。
- 知识驱动的链式思维 (Knowledge-Driven Chain-of-Thought, KD-CoT): 利用外部或背景知识引导模型进行推理。



- 交叉检索链式思维 (Interleaving Retrieval Chain-of-Thought): 将检索增强生成 (RAG) 技术融入推理链, 实时检索相关信息以辅助推理。
- 笔记链式思维 (Chain-Of-Note, CoN): 通过生成中间笔记记录推理过程, 帮助模型更系统化地回答问题。
- 知识链式思维 (Chain-Of-Knowledge, CoK): 集成知识推理的能力, 使模型能够在知识图谱或结构化信息的基础上推理。

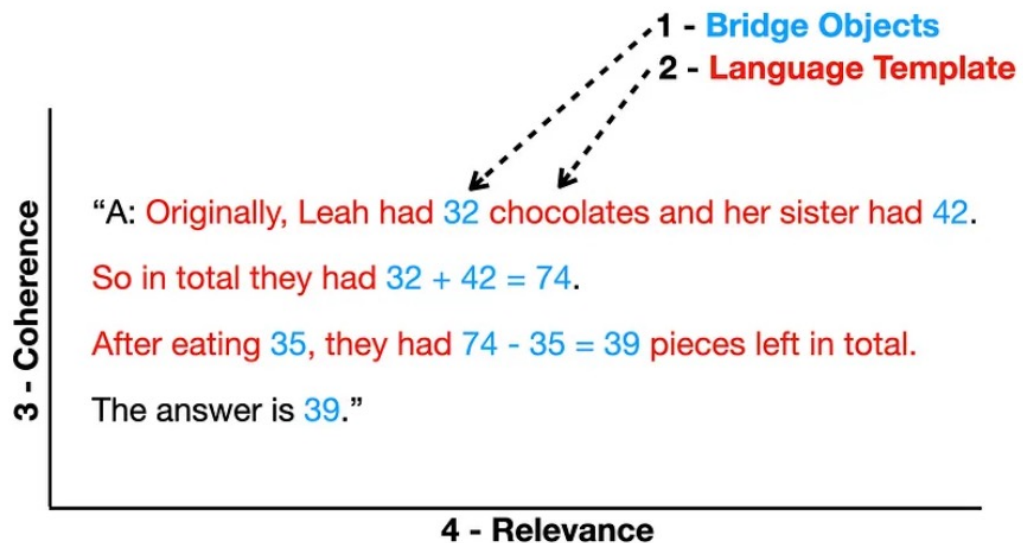
- 对比链式思维 (Contrastive Chain-Of-Thought, CCoT): 通过对比不同推理路径来评估最佳答案。
- 由LLM指导的树状思维 (LLM Guided Tree of Thought): 模型通过构建树状结构分析问题, 从不同角度探索可能的解答路径。
- 从易到难的提示设计 (Least-To-Most Prompting): 按照由简单到复杂的顺序逐步引导模型完成推理任务。

- 同理心链式思维 (Chain-Of-Empathy, CoE): 强调理解用户情感或语境, 通过共情能力提升模型的回答质量。
- 解释链式思维 (Chain-Of-Explanation, CoE): 注重模型对答案的详细解释, 增强结果的可解释性。



4 Key components of COT

4 Components of Chain-Of-Thought (CoT)



连贯性 (Coherence)

推理步骤按照正确的顺序排列

确保每一步推理是合理且具有因果关系的。
它帮助模型避免跳步或返回不相关的内容

桥接对象 (Bridge Objects)

“桥接对象”用来连接上下文，确保推理链条的逻辑连贯

数学任务：数字和方程式

事实推理任务：实体名称

模型在推理过程中需要 traverses (遍历) 的符号化元素

语言模板 (Language Template)

提供了一种结构化的语言框架

模型在推理过程中使用的文本提示，指导其推导正确的桥接对象并在推理过程中将其情境化

- 语言模板明确指引了每一步的逻辑结构和上下文。例如：
 - “Originally, Leah had 32 chocolates...” 确立初始条件。
 - “So in total they had...” 提示下一步是求和。
 - “After eating 35...” 引导进行减法运算。

相关性 (Relevance)

推理过程中包含的信息必须与问题直接相关，而不应该引入无关的内容或分支



Key Points of CoT Prompting

保持相关性是关键。（定方向）

对于桥接对象，相关性比连贯性更重要。

语言模板的连贯性至关重要。（定语言结构）

相关性和连贯性是提升CoT提示性能的关键。



相关性度量 @ COT

语义相似性评分

计算推理链条中的每一步与问题之间的语义相似度。

任务相关性评分 (Task-Specific Relevance)

训练一个分类器，基于人类标注的数据来判断推理步骤是否与问题相关。

信息增益 (Information Gain)

评估每一步推理对最终答案的贡献。



Relevance of bridging objects

桥接对象的重要性评分 (Object Importance Score)

桥接对象匹配率 (Bridging Object Matching Rate)

因果关系验证 (Causal Relationship Validation)

通过知识图谱或实体关系数据库验证桥接对象是否与问题和推理上下文直接相关



Coherence of Language Templates

逻辑顺序评分 (Logical Order Score)

语言流畅性评分 (Language Fluency Score)

上下文依赖性评分 (Context Dependency Score)

基于人类评价 (Human Evaluation) 的打分模型



COT 实现路径

问题分解 (Problem Decomposition)



上下文增强 (Context Enhancement)



提示设计 (Prompt Design)



推理链生成 (Chain Generation)



结果评估与优化 (Evaluation and Optimization)

```
def chain_of_thought(prompt, knowledge_base, max_steps=5, .
    """
    实现 Chain-of-Thought (CoT) 推理路径
    :param prompt: 用户输入的问题
    :param knowledge_base: 外部知识库或上下文检索工具
    :param max_steps: 最大推理步骤
    :return: 最终答案及推理链
    """
    # 初始化推理链
    reasoning_chain = []
    current_prompt = prompt

    # 第 1 步: 问题分解
    decomposed_problems = decompose_problem(prompt)

    # 第 2 步: 增强上下文
    for sub_problem in decomposed_problems:
        # 检索相关上下文
        context = retrieve_context(sub_problem, knowledge_base)
        current_prompt += f"\nContext: {context}"

    # 第 3 步: 提示设计
    cot_prompt = (
        f"Using the following context, answer step-by-step:\n"
        f"Problem: {sub_problem}\n"
        f"{current_prompt}\nStep-by-step Reasoning:"
    )

    # 第 4 步: 生成推理链
    reasoning_step = generate_text(cot_prompt)
    reasoning_chain.append(reasoning_step)

    # 第 5 步: 结果评估
    if not evaluate_relevance(reasoning_step, sub_problem) or n
        print("Warning: Low quality reasoning step detected!")
        break

    # 如果达到最大推理步数, 停止
    if len(reasoning_chain) >= max_steps:
        break

    # 汇总结果
    final_answer = reasoning_chain[-1] if reasoning_chain else "Una
    return final_answer, reasoning_chain
```



Key take away

数据是链式思维性能提升的核心，正确的数据直接影响模型推理的

数据发现 (Data Discovery) :
确定需要的关键数据。

数据设计 (Data Design) :
设计结构化且高效的数据集。

数据开发与交付 (Data Development & Delivery) :
确保数据的生成和使用符合目标需求。

RAG meets LLM

(retrieval augmented Generation)

Integrating IR in Generation : RA-LLM

Data for Training LLMs

- Low quality
- General
- Fixed
- Hard to update

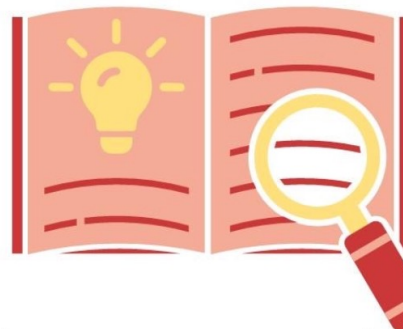


Content generation

*Close-book exam
(Hard mode, have to
remember everything)*

External Knowledge Base

- High-quality knowledge
- Specialized knowledge
- Scalable
- Easy-updated



Information / Knowledge
retrieval

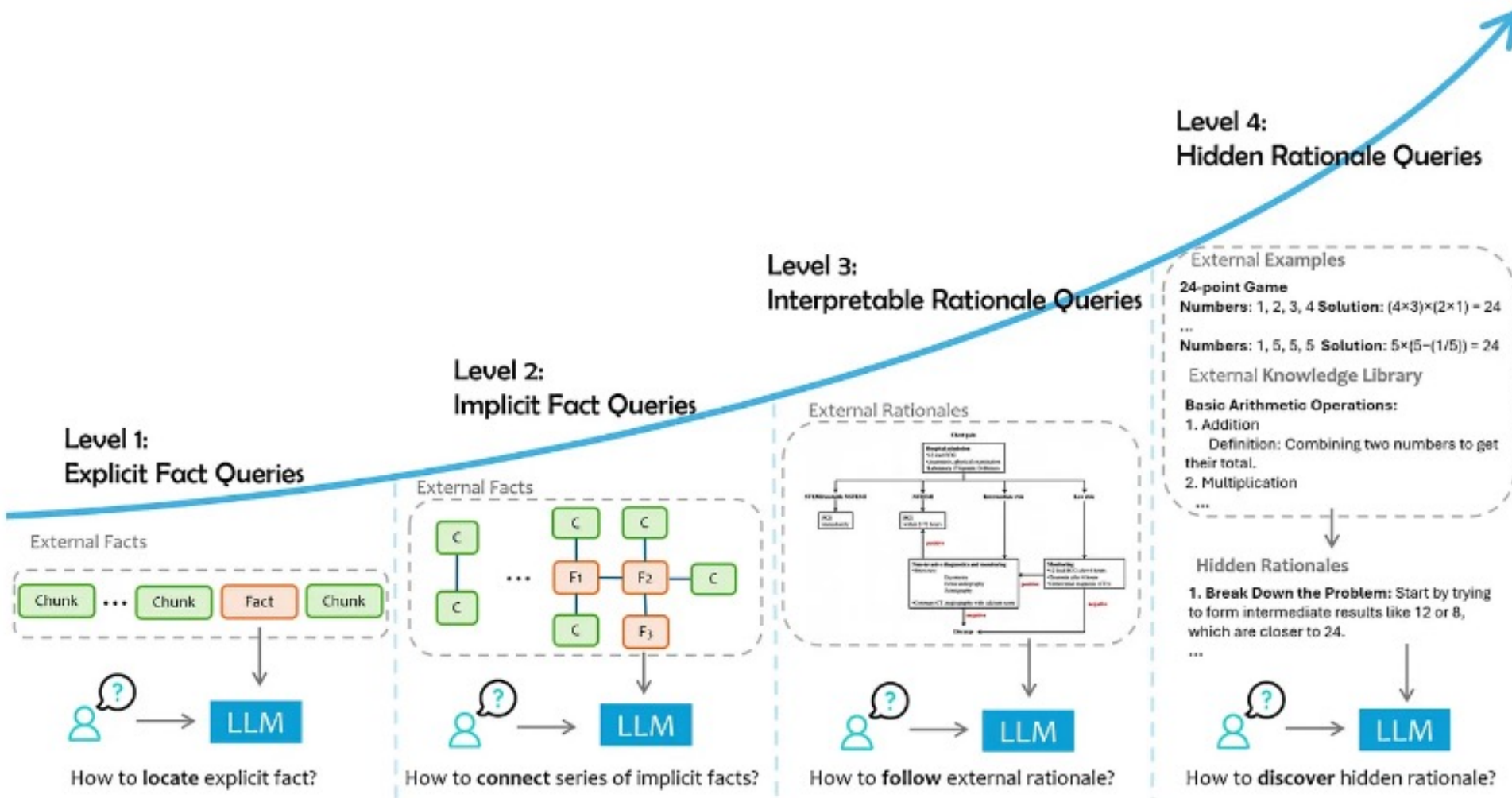
=



RA-LLMs

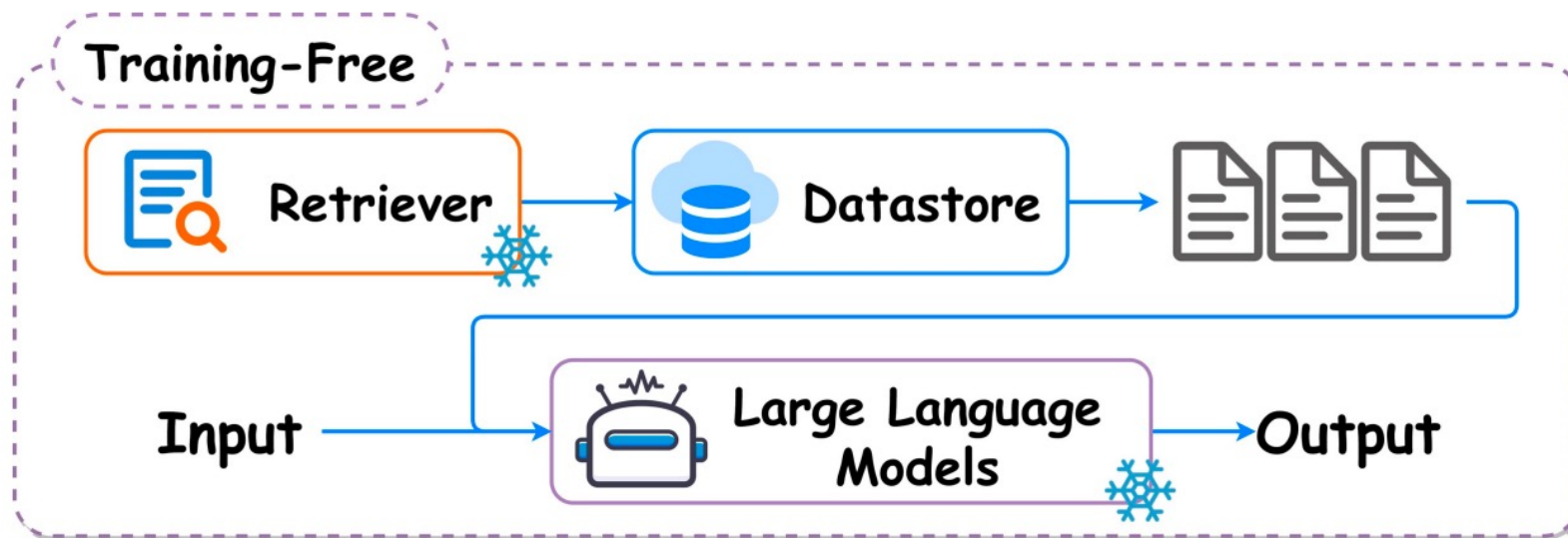
*Open-book exam
(Easy mode, allow to search
in reference)*

Four levels of RAG



RA-LLM learning : training free

Retrieval models and language models are both frozen.

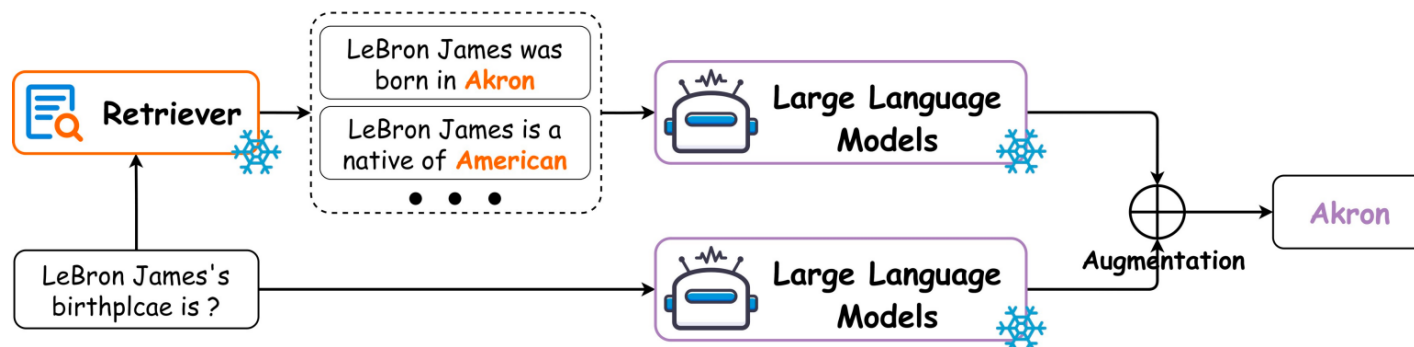


RA-LLM learning : training free

Prompt Engineering-based Methods



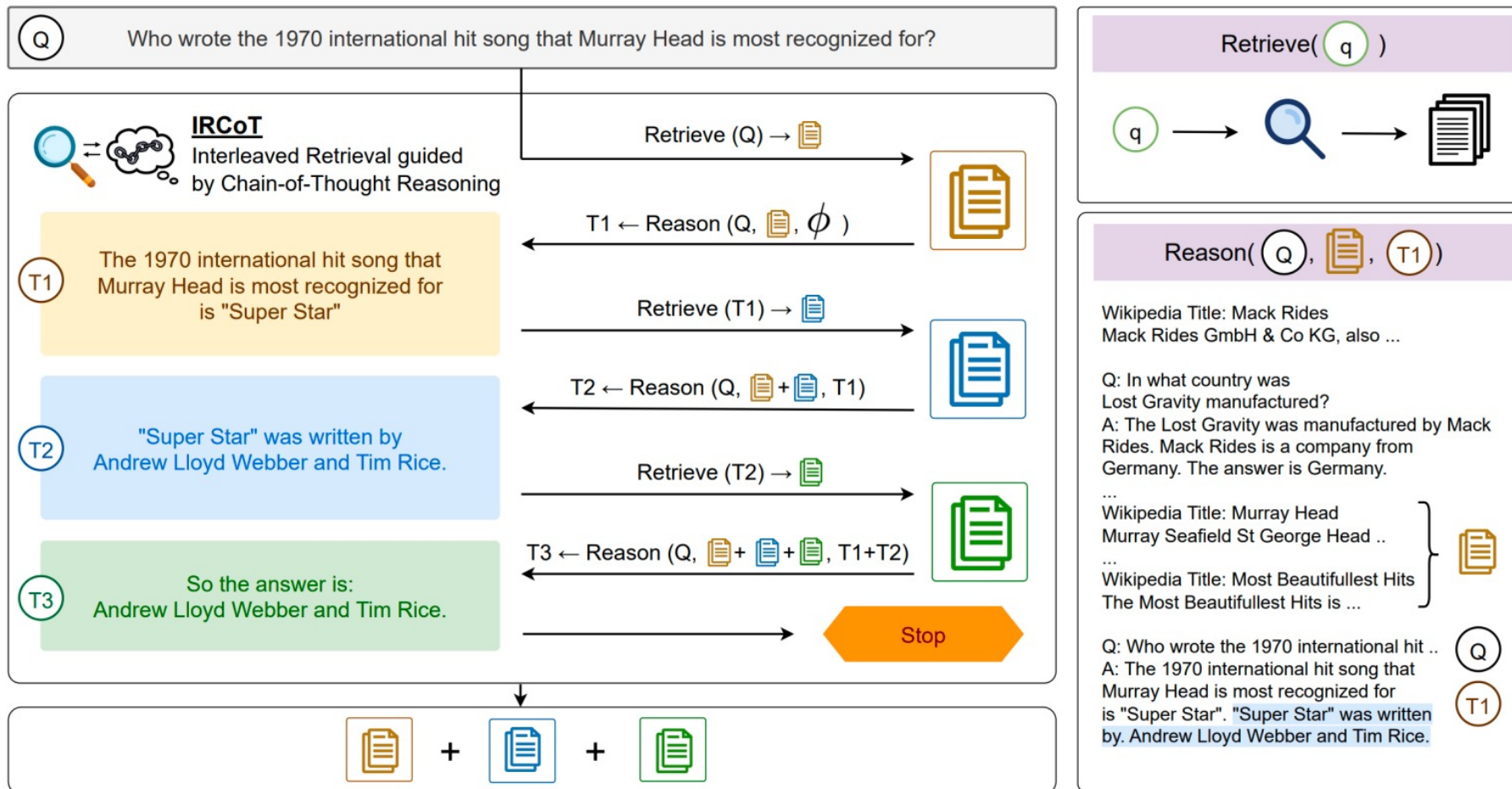
Retrieval-Guided Token Generation Methods





RA-LLM learning : training free

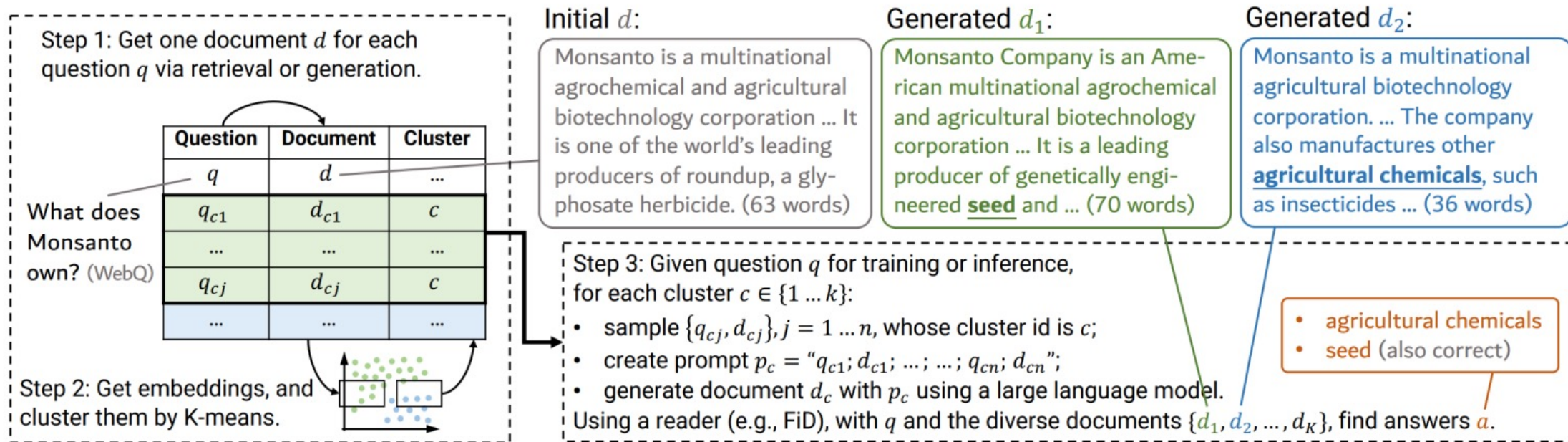
IRCoT (Interleaved Retrieval guided by Chain-of-Thought Reasoning, 基于链式思维推理的交替检索)



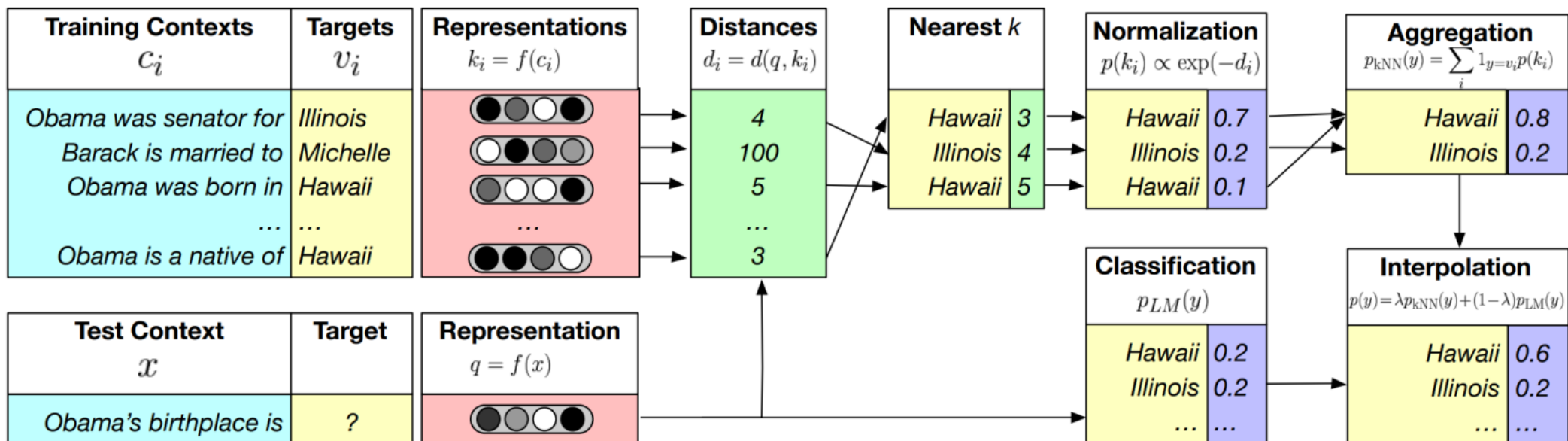


RA-LLM learning : training free

通过检索或生成多样化的文档，结合聚类 and 嵌入的方法来生成准确答案



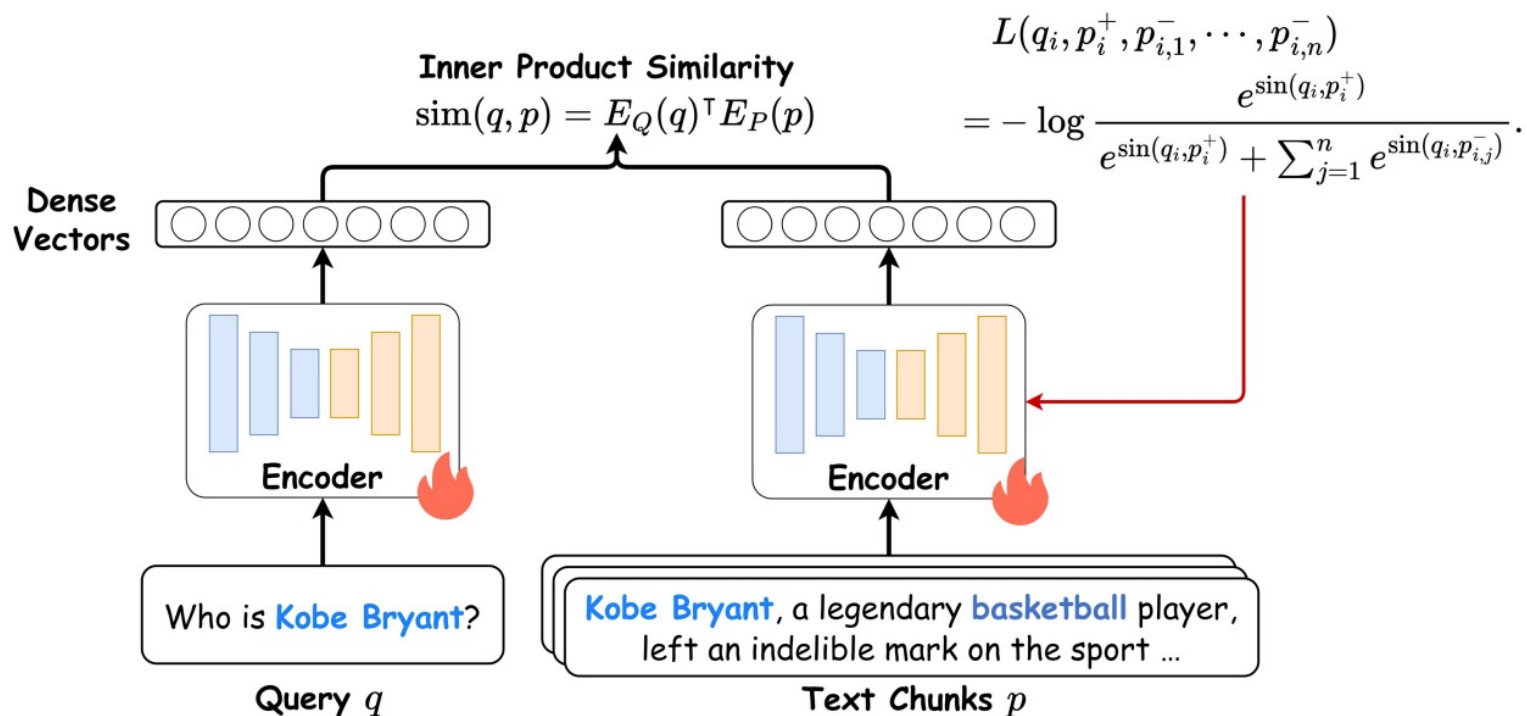
RA-LLM learning : training free



$$p(y|x) = \lambda p_{KNN}(y|x) + (1 - \lambda) p_{LM}(y|x)$$

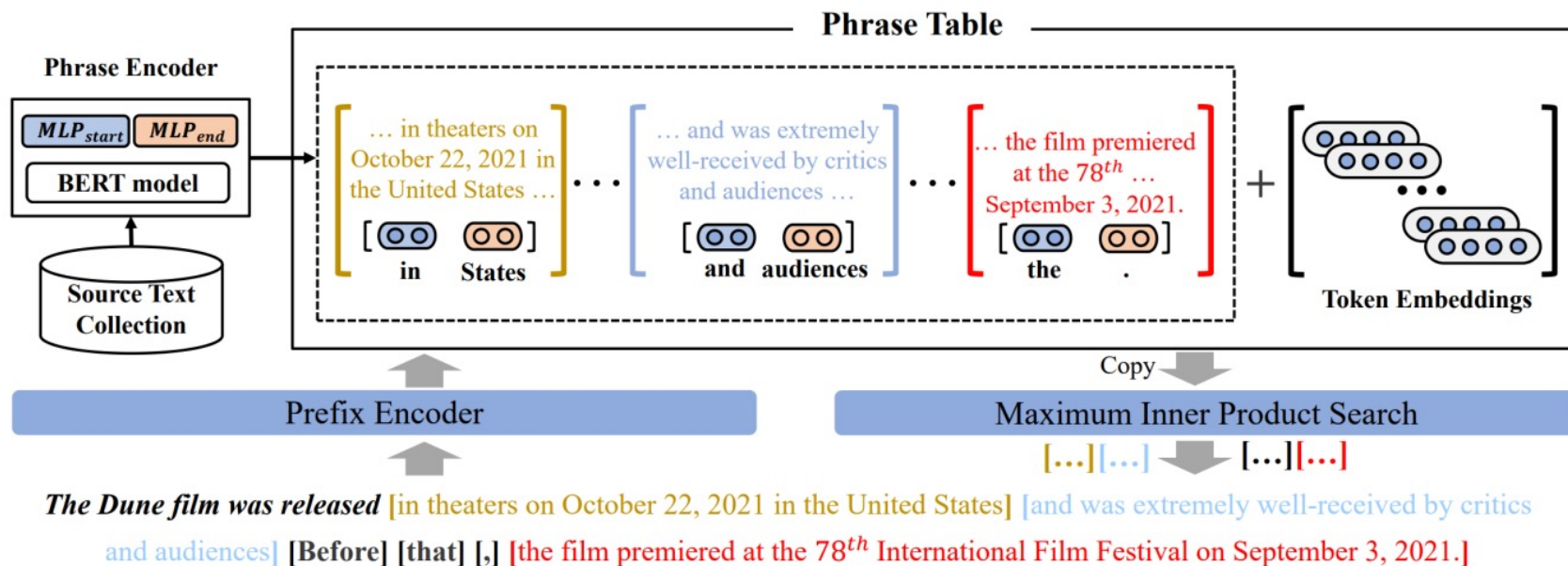
RA-LLM learning : independent training

- Dense retrieval models: DPR



RA-LLM learning : independent training

- Dense retrieval models: CoG



$$\mathcal{D}_{start} = MLP_{start}(\mathcal{D}), \mathcal{D}_{end} = MLP_{end}(\mathcal{D}).$$

$$\mathcal{H}_{i+1} = PrefixEncoder(x_i, \mathcal{H}_i).$$

$$PhraseEncoder(s, e, D) = [\mathcal{D}_{start}[s]; \mathcal{D}_{end}[e]] \in \mathbb{R}^d$$



RA-LLM learning : independent training

短语匹配损失

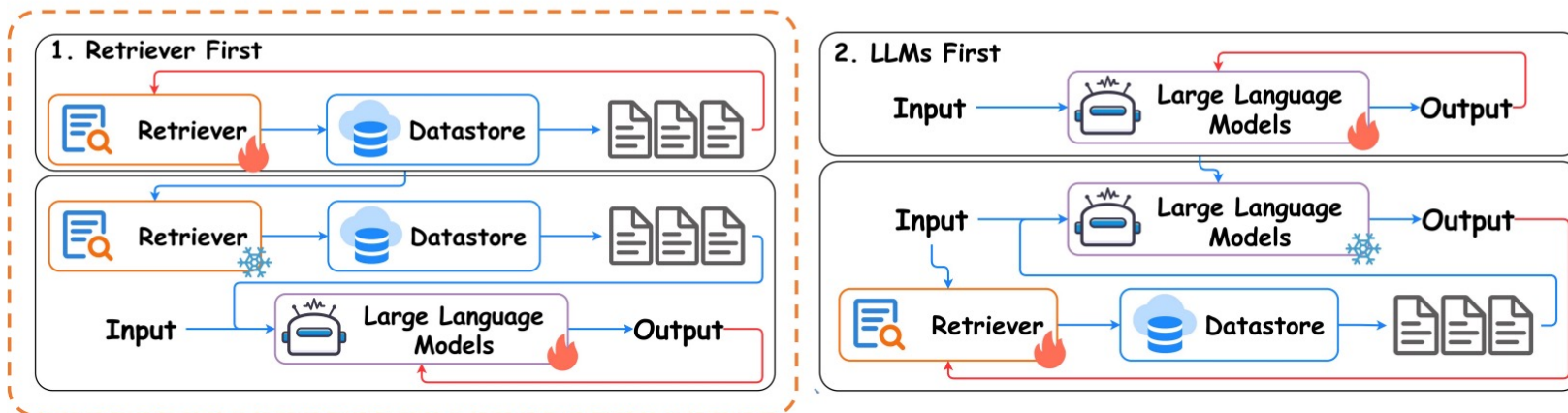
$$\mathcal{L}_p = -\frac{1}{n} \sum_{k=1}^n \log \frac{\exp(q_k \cdot p_k)}{\sum_{p \in \mathcal{P}_k} \exp(q_k \cdot p_p) + \sum_{w \in V} \exp(q_k \cdot v_w)}$$

$$\mathcal{L}_t = -\frac{1}{m} \sum_{i=1}^m \log \frac{\exp(q_i, v_{D_i})}{\sum_{w \in V} \exp(q_i, v_w)}$$

目标分类损失

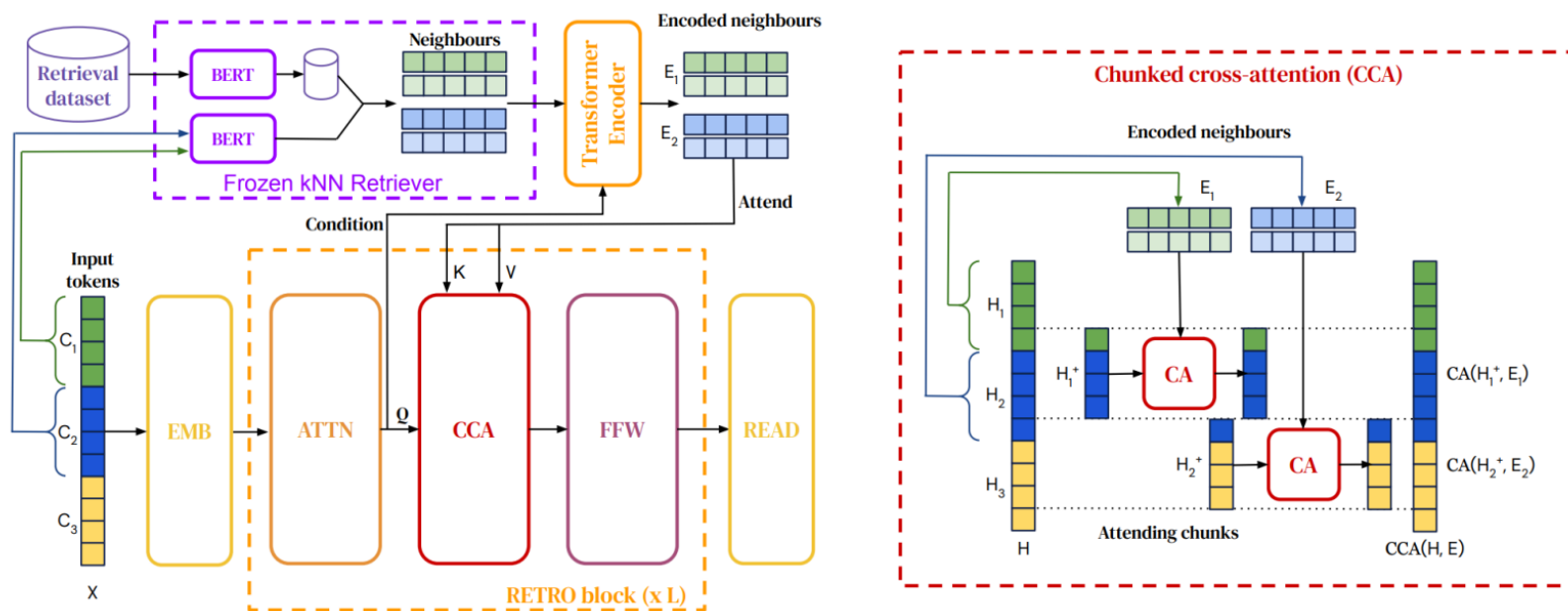
RA-LLM learning : sequence training

- **Retrieval models** is first trained independently and then fixed.
- **Language models** are trained with an objective that depends on **the Retrieval**.



RA-LLM learning : sequence training

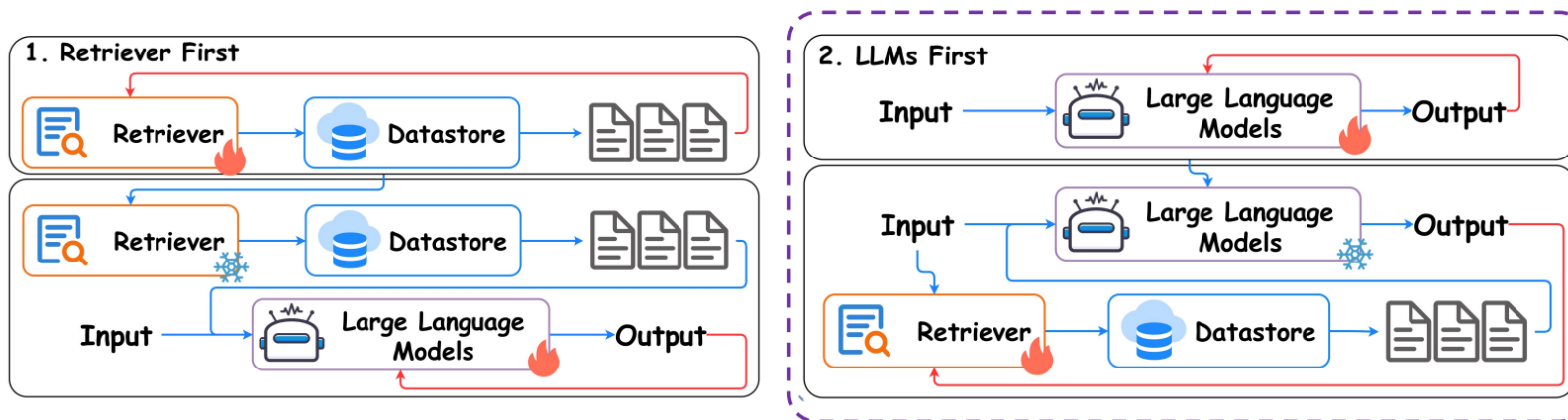
- RETRO



将检索模块与 Transformer 模型结合的架构，用于增强语言生成和理解任务。其核心思想是通过检索相关文档或上下文邻居来辅助语言模型更好地生成或理解文本

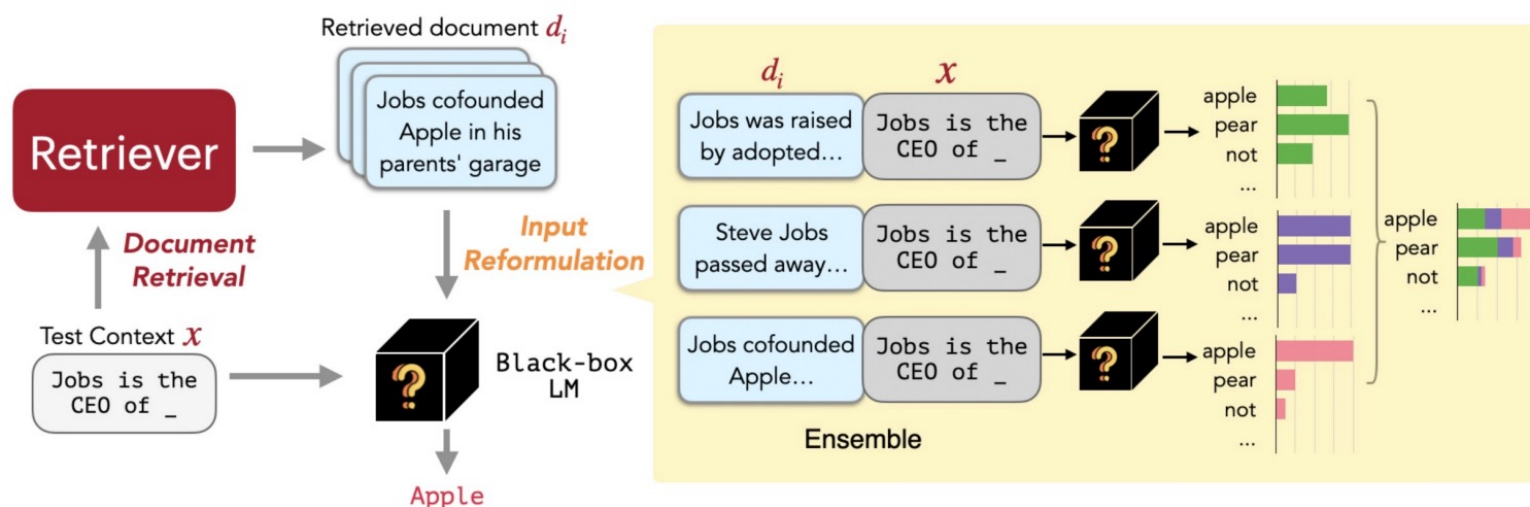
RA-LLM learning : sequence training

- **Language models** are first trained independently and then fixed.
- **Retrieval models** are trained with supervisions from **language models**.



RA-LLM learning : sequence training

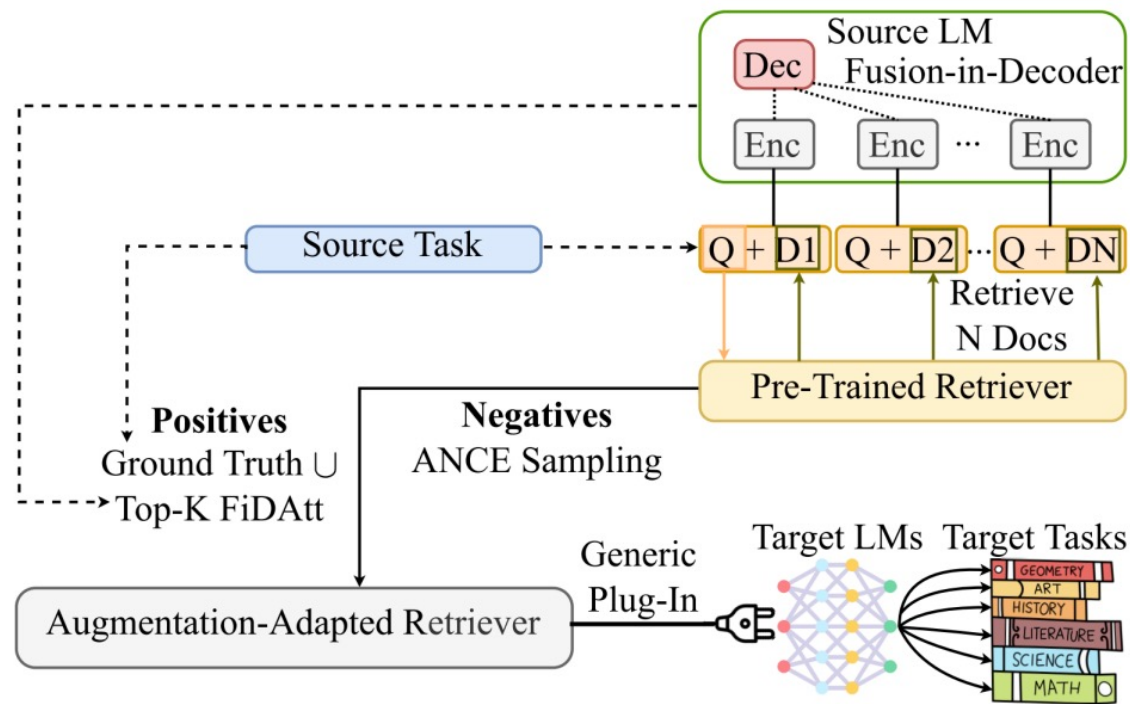
- REPLUG (Retrieve and Plug)



$$\mathcal{L} = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} KL(P_R(d | x) \parallel Q_{LM}(d | x, y)) \quad P_R(d | x) = \frac{e^{s(d, x)/\gamma}}{\sum_{d \in \mathcal{D}'} e^{s(d, x)/\gamma}} \quad Q(d | x, y) = \frac{e^{P_{LM}(y|d, x)/\beta}}{\sum_{d \in \mathcal{D}'} e^{P_{LM}(y|d, x)/\beta}}$$

RA-LLM learning : sequence training

- AAR (Augmentation-Adapted Retriever)**



$$\mathcal{L} = \sum_q \sum_{d^+ \in D^+} \sum_{d^- \in D^-} l(f(q, d^+), f(q, d^-)),$$

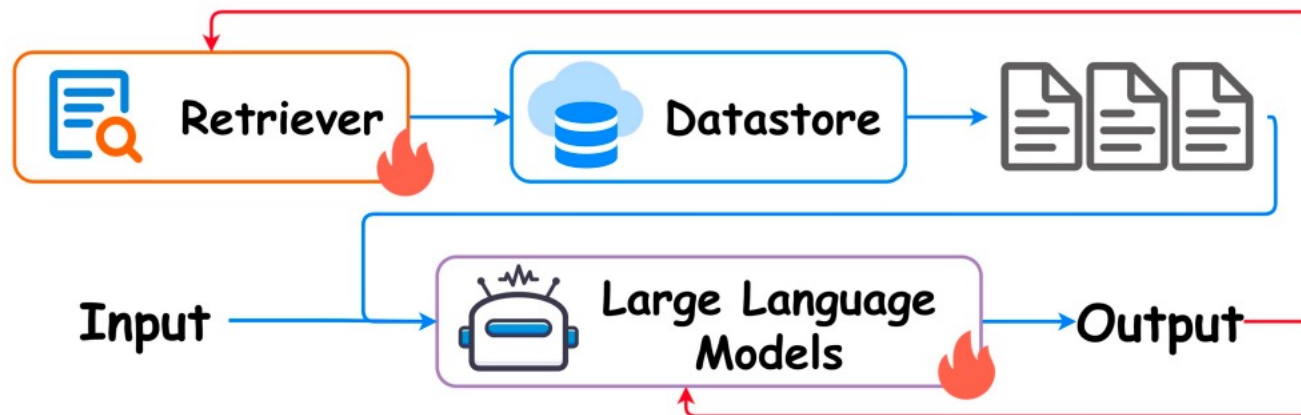


RA-LLM learning : sequence training

- ✓ Work with off-the-shelf models
- ✓ Generators can be trained effectively based on the retrieved results
- ✓ Retrievers can be trained to provide useful information to help the generators
- x One component is still fixed and not trained
- x Might not achieve optimal learning result of the whole model

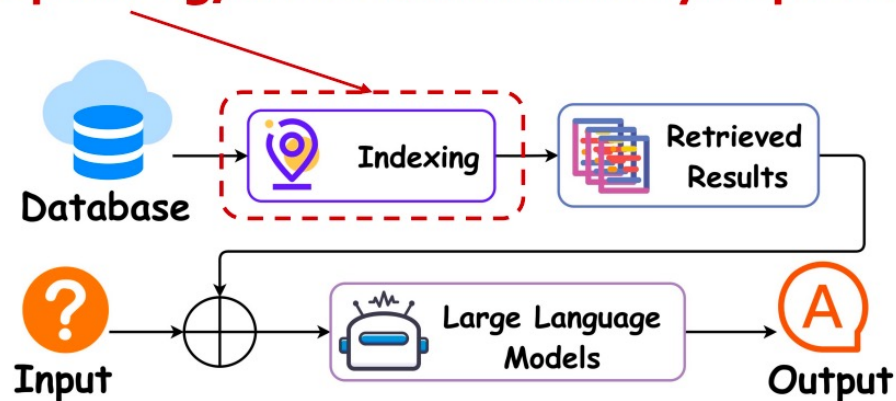
RA-LLM learning : joint training

Retrieval models and language models are trained jointly



RA-LLM learning : joint training

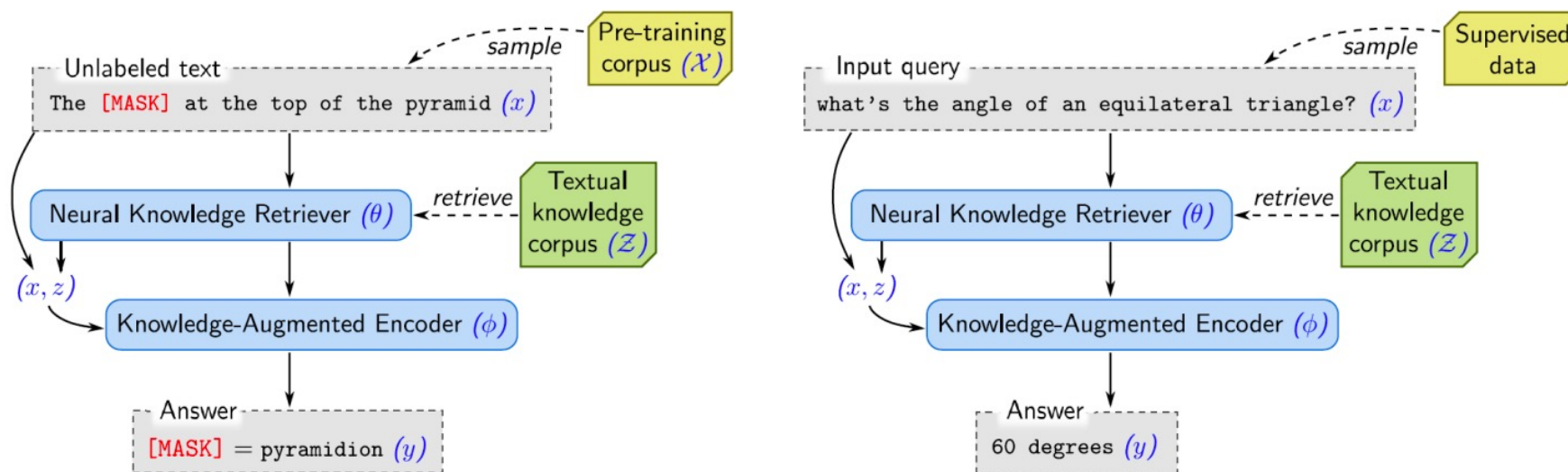
- **Retrieval Index Updating, which could be very expensive!**



- **Solutions:**
 - Asynchronous index updating
 - In-batch approximation

RA-LLM learning : joint training

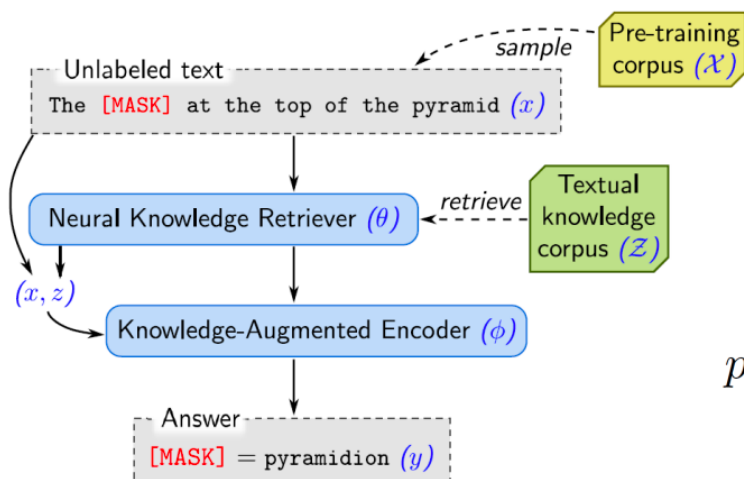
- REALM



Objective function:
$$p(y | x) = \sum_{z \in \mathcal{Z}} p(y | z, x) p(z | x).$$

RA-LLM learning : joint training

- REALM

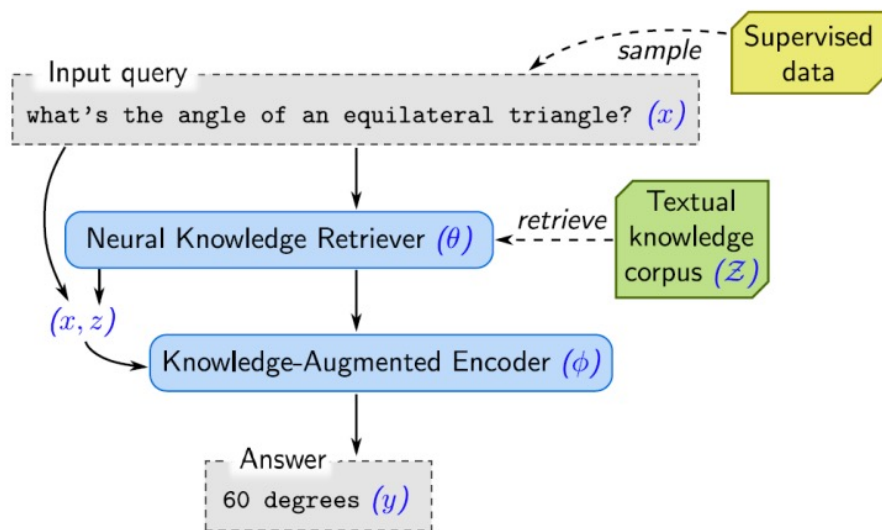


$$p(y | z, x) = \prod_{j=1}^{J_x} p(y_j | z, x)$$

$$p(y_j | z, x) \propto \exp(w_j^\top \text{BERT}_{\text{MASK}(j)}(\text{join}_{\text{BERT}}(x, z_{\text{body}})))$$

RA-LLM learning : joint training

- REALM



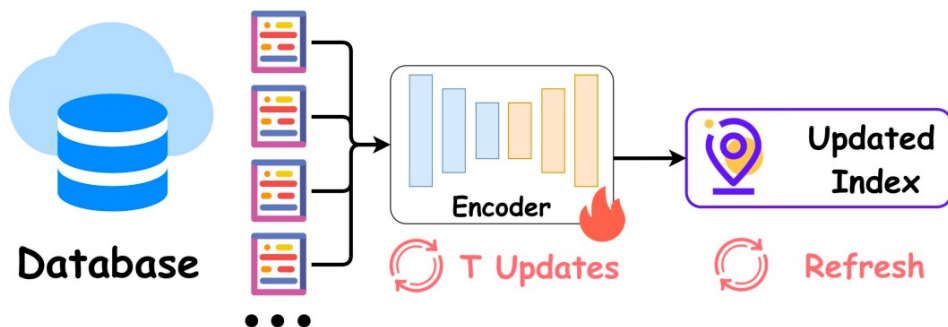
$$p(y | z, x) \propto \sum_{s \in S(z, y)} \exp(\text{MLP}([h_{\text{START}(s)}; h_{\text{END}(s)}]))$$

$$h_{\text{START}(s)} = \text{BERT}_{\text{START}(s)}(\text{join}_{\text{BERT}}(x, z_{\text{body}})),$$

$$h_{\text{END}(s)} = \text{BERT}_{\text{END}(s)}(\text{join}_{\text{BERT}}(x, z_{\text{body}})),$$

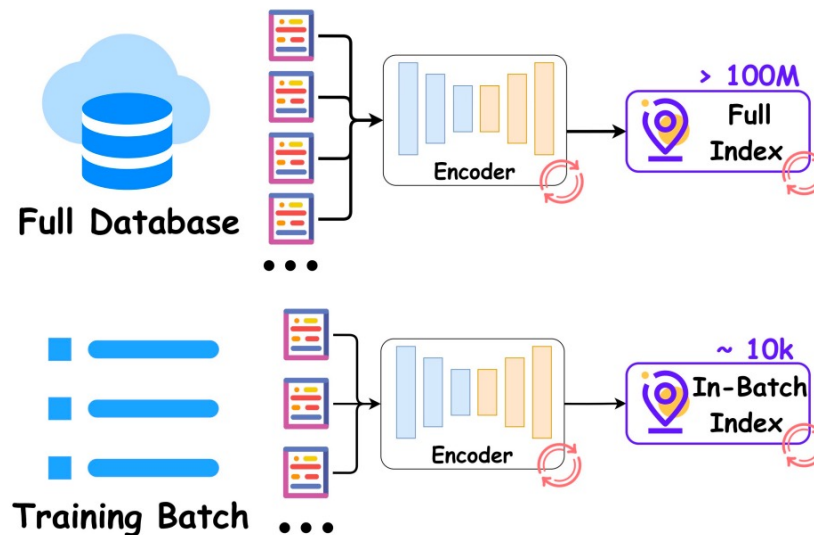
RA-LLM learning : joint training

- REALM – Asynchronous Index Update



$$f(x, z) = \text{Embed}_{\text{input}}(x)^{\top} \text{Embed}_{\text{doc}}(z)$$

- TRIME – In-Batch Approximation

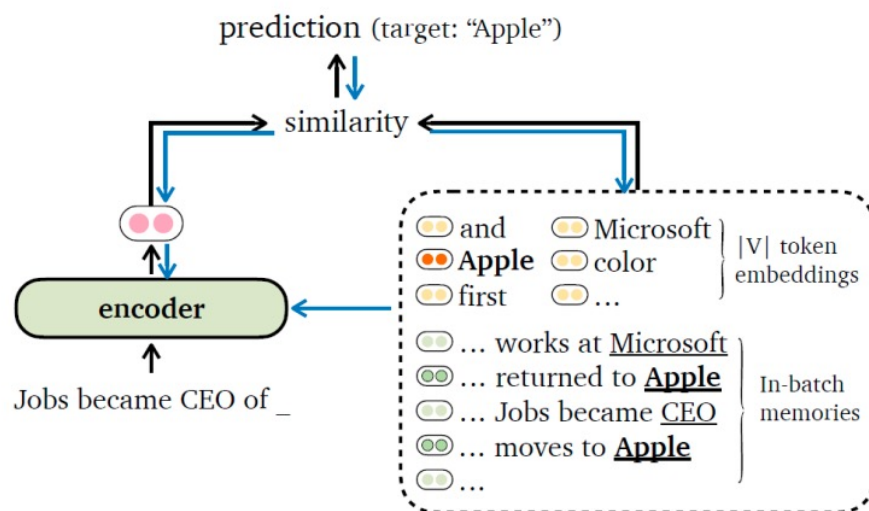


RA-LLM learning : joint training

• TRIME

- Target token's embedding
- Other token embeddings
- Positive in-batch memory
- Negative in-batch memory

↑ Forward pass ↓ Back-propagation



Local Memory: $\mathcal{M}_{\text{local}}(c_t) = \{(c_j, x_j)\}_{1 \leq j \leq t-1}$.

Long-term Memory:

$$\mathcal{M}_{\text{long}}(c_t^{(i)}) = \{(c_j^{(k)}, x_j^{(k)})\}_{1 \leq k < i, 1 \leq j}$$

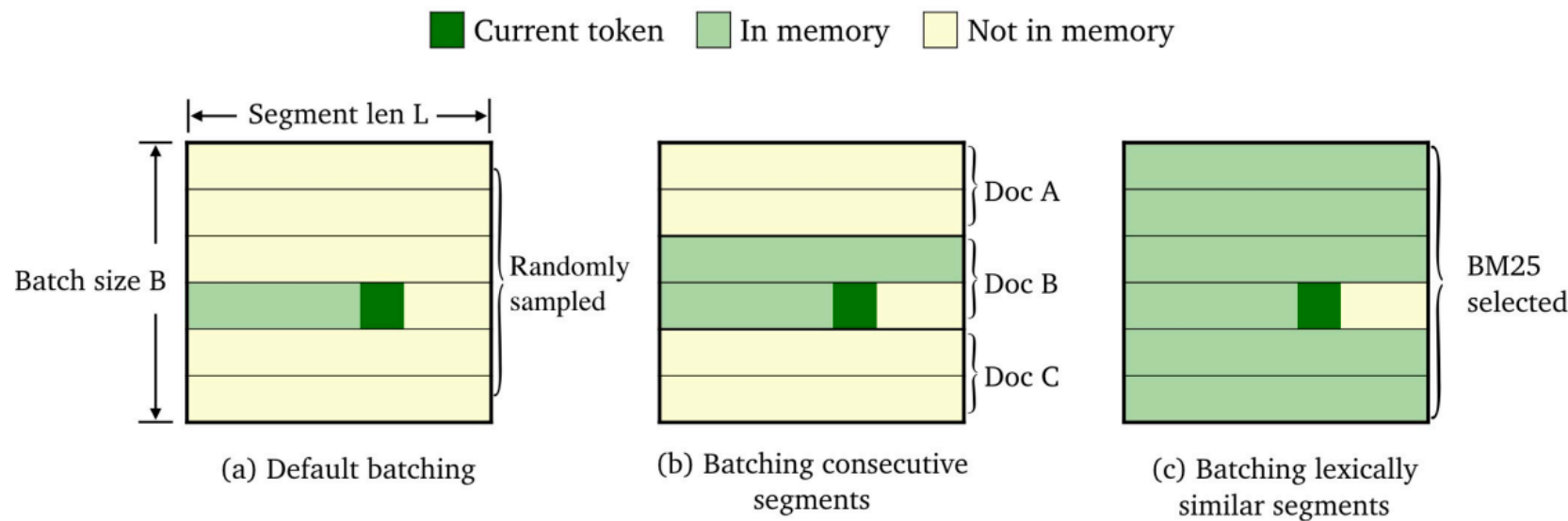
External Memory: $\mathcal{M}_{\text{ext}} = \{(c_j, x_j) \in \mathcal{D}\}$.

Training Objective:

$$P(w | c) \propto \exp(E_w^\top f_\theta(c)) + \sum_{(c_j, x_j) \in \mathcal{M}_{\text{train}}: x_j = w} \exp(\text{sim}(g_\theta(c), g_\theta(c_j))).$$

RA-LLM learning : joint training

- TRIME Data Batching Strategy



Use BM25 scores to find similar text chunks to provide more training signals

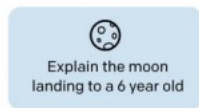
Reinforcement Learning from human relevance feedback

RLHF – reinforcement learning from human feedback

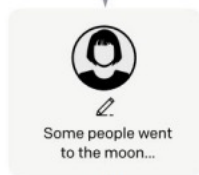
Step 1

Collect demonstration data, and train a supervised policy.

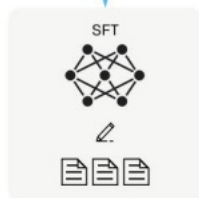
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



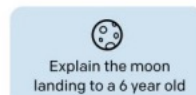
This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

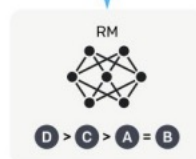
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.



The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.





RLHF – reward model

Feedback comes as **preferences over model samples**: $\mathcal{D} = \{x^i, y_w^i, y_l^i\}$

Prompt $\nearrow$ Preferred response $\nwarrow$ Dispreferred response

Bradley-Terry Model connects rewards to preferences:

Reward assigned to **preferred** and **dispreferred** responses

$$p(y_w \succ y_l \mid x) = \sigma(r(x, y_w) - r(x, y_l))$$

Train the reward model by **minimizing negative log likelihood**:

$$\mathcal{L}_R(\phi, \mathcal{D}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$$



RLHF – learning a policy that optimizes the reward

Now we have a **reward model** r_ϕ that represents* **goodness according to humans**

Now, learn a policy π_θ achieving **high reward** while **staying close** to original model π_{ref}

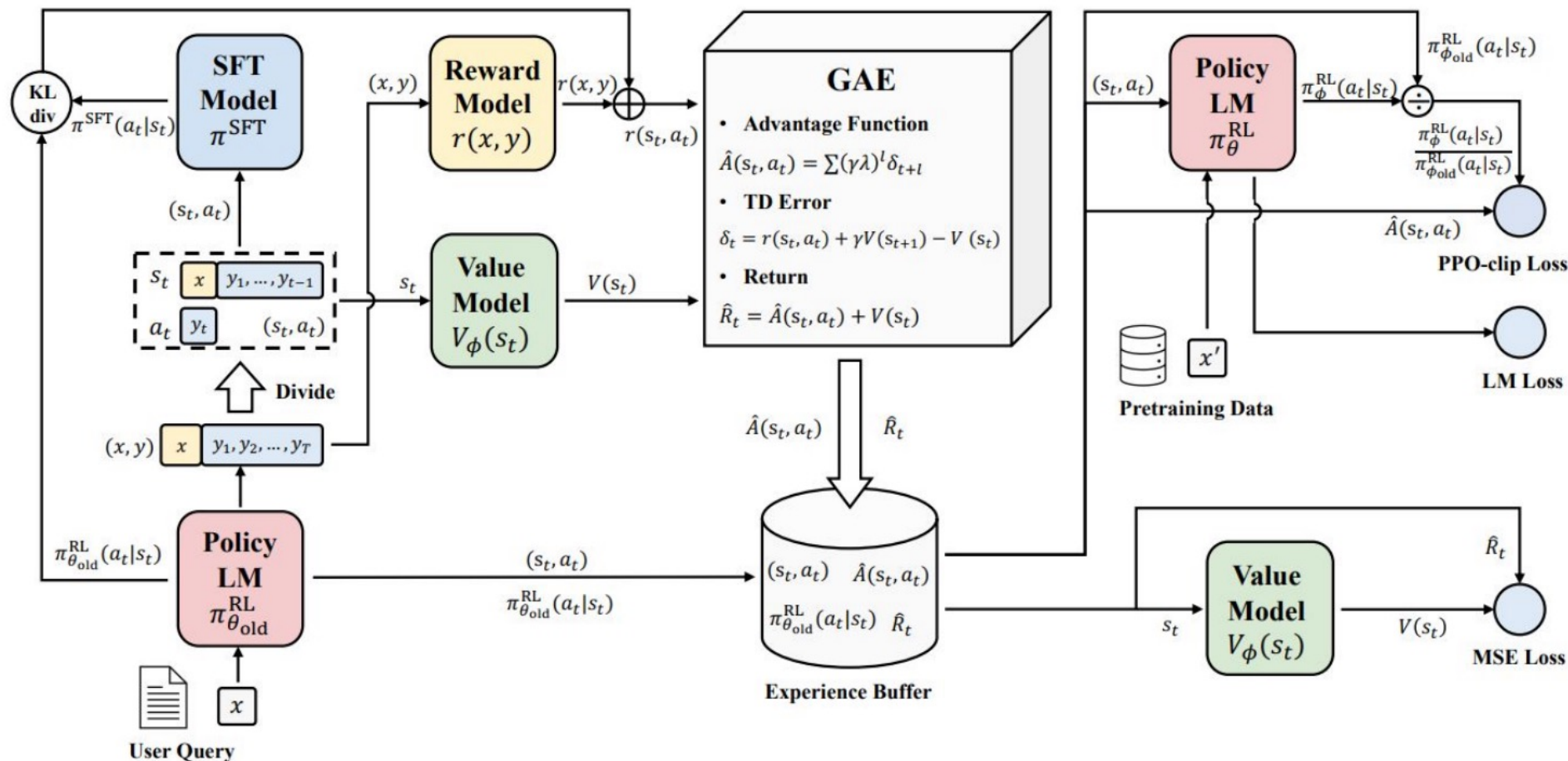
$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)} [r_\phi(x, y)] - \beta \mathbb{D}_{\text{KL}}[\pi_\theta(y|x) || \pi_{\text{ref}}(y|x)]$$

Sample from policy

Want high reward...

...but keep KL to original model small!

RLHF – learning a policy that optimizes the reward





RLHF – Direct Preference Optimization

RLHF Objective

(get **high reward**, stay **close** to reference model)

$$\max_{\pi} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi(y|x)} [r(x, y)] - \beta \mathbb{D}_{\text{KL}}(\pi(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x))$$

← **any** reward function

Closed-form Optimal Policy

(write **optimal policy** as function of **reward function**; from prior work)

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y | x) \exp \left(\frac{1}{\beta} r(x, y) \right)$$

$$\text{with } Z(x) = \sum_y \pi_{\text{ref}}(y | x) \exp \left(\frac{1}{\beta} r(x, y) \right)$$

← Note **intractable sum** over possible responses; can't immediately use this

Rearrange

(write **any reward function** as function of **optimal policy**)

$$r(x, y) = \underbrace{\beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x)}_{\text{some parameterization of a reward function}}$$



RLHF – Direct Preference Optimization

A loss function on reward functions

+

A transformation between reward functions and policies

=

A loss function on policies

Derived from the Bradley-Terry model of human preferences:

$$\mathcal{L}_R(r, \mathcal{D}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r(x, y_w) - r(x, y_l))]$$

$$r_{\pi_\theta}(x, y) = \beta \log \frac{\pi_\theta(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x)$$

When substituting, the **log Z term cancels**, because the loss only cares about **difference** in rewards

Reward of
preferred
response

Reward of
dispreferred
response

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$



RLHF – Direct Preference Optimization

Quiet Self-Taught Reasoner 安静的自学推理者

Quiet-STaR: Language Models Can Teach Themselves to Think Before Speaking

Eric Zelikman
Stanford University

Georges Harik
Notbad AI Inc

Yijia Shao
Stanford University

Varuna Jayasiri
Notbad AI Inc

Nick Haber
Stanford University

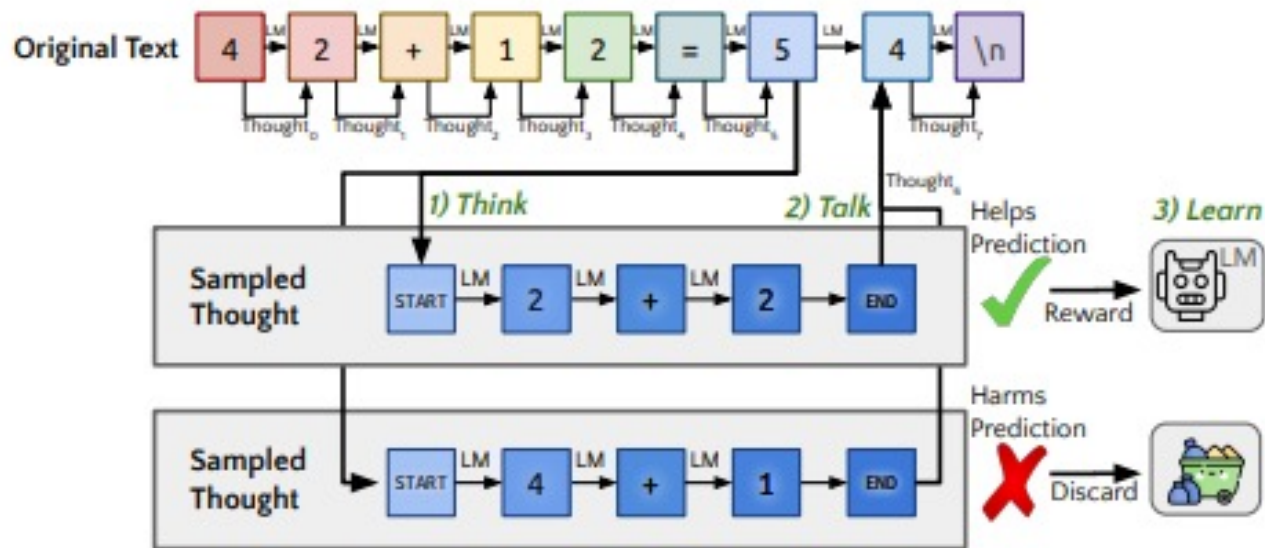
Noah D. Goodman
Stanford University

Abstract

When writing and talking, people sometimes pause to think. Although reasoning-focused works have often framed reasoning as a method of answering questions or completing agentic tasks, reasoning is implicit in almost all written text. For example, this applies to the steps not stated between the lines of a proof or to the theory of mind underlying a conversation. In the Self-Taught Reasoner (STaR, Zelikman et al. 2022), useful thinking is learned by inferring rationales from few-shot examples in question-answering and learning from those that lead to a correct answer. This is a highly constrained setting – ideally, a language model could instead learn to infer unstated rationales in arbitrary text. We present **Quiet-STaR**, a generalization of STaR in which LMs learn to generate rationales at each token to explain future text, improving their predictions. We address key challenges, including 1) the computational cost of generating continuations, 2) the fact that the LM does not initially know how to generate or use internal thoughts, and 3) the need to predict beyond individual next tokens. To resolve these, we propose a tokenwise parallel sampling algorithm, using learnable tokens indicating a thought’s start and end, and an extended teacher-forcing technique. Encouragingly, generated rationales disproportionately help model difficult-to-predict tokens and improve the LM’s ability to directly answer difficult questions. In particular, after continued pretraining of an LM on a corpus of internet text with Quiet-STaR, we find zero-shot improvements on GSM8K (5.9%→10.9%) and CommonsenseQA (36.3%→47.2%) and observe a perplexity improvement of difficult tokens in natural text. Crucially, these improvements require no fine-tuning on these tasks. Quiet-STaR marks a step towards LMs that can learn to reason in a more general and scalable way.

“Life can only be understood backwards; but it must be lived forwards.”

— Sren Kierkegaard



思考：

针对每一个输入文本序列，模型会生成多种可能的“推理过程”或“中间思路”，称为**采样思路 (Sampled Thoughts)**

表达：

这些采样的思路被用来预测下一个目标
模型在训练时会同时利用有思路的预测路径和无思路的预测路径来进行计算

学习：

模型通过奖励机制 (如 **REINFORCE**，一种强化学习算法) 对每个“思路”的贡献进行评估

生成推理路径 T :

给定输入 X , 生成推理路径 T , 其概率分布为:

$$P(T|X) = \text{softmax}(f_{\text{LM}}(X))$$

其中 f_{LM} 是语言模型的推理生成函数。

基于推理路径生成目标预测 Y :

结合 T 和 X , 生成目标预测的条件概率:

$$P(Y|X, T) = \prod_{t=1}^m P(y_t|X, T, y_{<t})$$

优化目标:

模型通过强化学习优化以下目标:

$$\mathcal{L} = -\mathbb{E}_{T \sim P(T|X)} \left[\log P(Y|X, T) \right]$$

- T 是动态采样生成的推理路径。
- 强化学习的奖励机制用于调整 T 的生成策略。

并行采样算法

Quiet-STaR 提出了一个**并行采样算法**, 能够从每个位置生成潜在的推理路径 (rationales), 以便提高训练效率和扩展性。

- 实现:
 - 对于每个输入序列 X , 模型并行地生成多个推理路径 $\{T_1, T_2, \dots, T_k\}$, 分别对应不同的推理假设。
 - 每个路径的生成是通过一个概率模型 $P(T|X)$ 实现的。
- 数学表达:
 - 并行采样的生成公式为:

$$T_k \sim P(T_k|X, \theta)$$

其中 θ 是模型参数, 通过最大化 $P(Y|X, T_k)$ 的期望来优化:

$$\mathcal{L}_{\text{parallel}} = -\sum_{k=1}^K \mathbb{E}_{T_k} \left[\log P(Y|X, T_k) \right]$$

引入 Meta-Tokens

为了明确区分“推理”和“预测”两个阶段，Quiet-STaR 在每个推理路径的开头和结尾引入了自定义的元标记 (meta-tokens)。

- 目的：
 - 让模型学习到哪些内容属于推理，哪些内容属于预测，并通过这些标记指导模型生成更准确的推理。
- 数学表示：
 - 对于推理路径 T ，在其开头和结尾分别添加标记 [START_THOUGHT] 和 [END_THOUGHT]，最终路径表示为：
$$T = \{[\text{START_THOUGHT}], t_1, t_2, \dots, t_k, [\text{END_THOUGHT}]\}$$
 - 模型优化目标变为：
$$\mathcal{L}_{\text{meta}} = -\log P([\text{END_THOUGHT}]|X, T, [\text{START_THOUGHT}])$$

Why does ice float on water? [START_THOUGHT]
Ice floats because its density is lower than that of liquid water. The hydrogen bonds in ice create a lattice structure that occupies more space, reducing its density. [END_THOUGHT] The final answer is:

混合头 (Mixing Head)

引入混合头以动态整合基于推理路径 T 的预测和直接基于输入 X 的预测，最终决定生成的下一个 Token。

- 目的：
 - 在预测过程中，根据当前生成的推理 T 的质量动态调整对最终预测的贡献。
- 数学表达：
 - 混合预测的公式为：

$$P(y_t|X) = \alpha P(y_t|X, T) + (1 - \alpha) P(y_t|X)$$

其中 $\alpha \in [0, 1]$ 是混合权重，依据 T 的贡献动态调整。

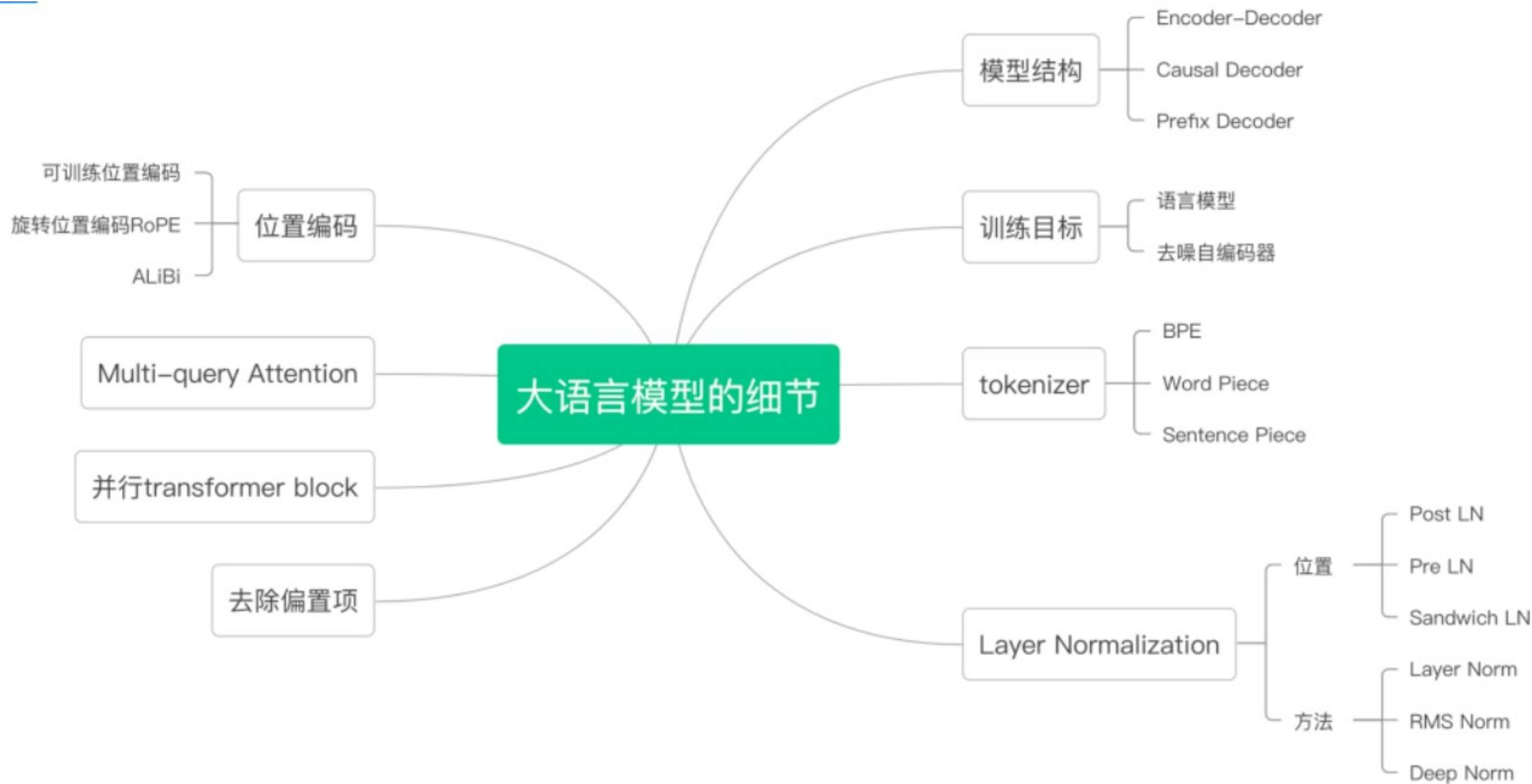
非短视损失函数 (Non-Myopic Loss)

传统语言模型关注单步预测，而 Quiet-STaR 使用**多步前瞻损失函数**，将未来多个 Token 的影响纳入考虑。

- 目的：
 - 不仅优化当前 Token 的预测准确性，还提升长远语境的一致性。
- 数学表达：
 - 非短视损失函数优化公式：

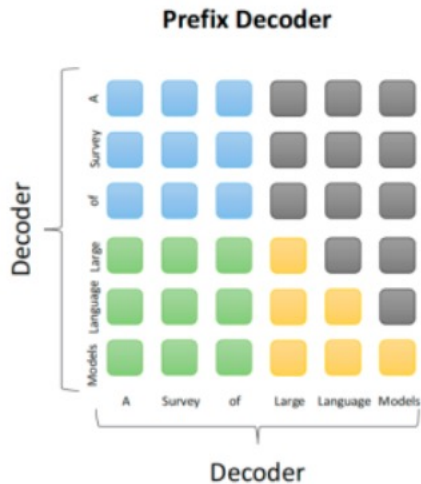
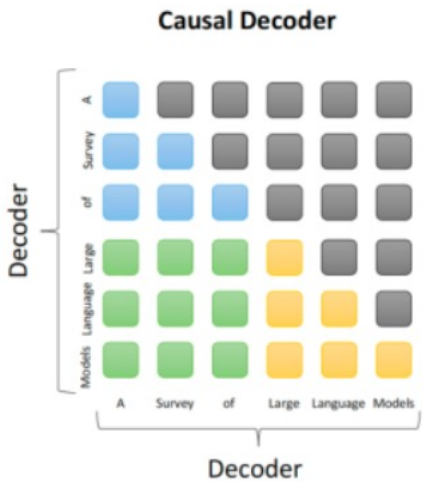
$$\mathcal{L}_{\text{non-myopic}} = - \sum_{t=1}^m \mathbb{E}_T \left[\sum_{j=0}^N \log P(y_{t+j} | X, T) \right]$$

其中 N 是前瞻步数，表示模型在训练中考虑了未来 N 个 Token 的生成质量。





模型结构



模型结构	结构特点	优势	劣势	适用场景
Encoder-Decoder	输入：双向注意力 输出：单向注意力	理解能力强，适合需要深入理解任务	训练效率低，长文本生成能力较差	翻译、问答、摘要生成等
Causal Decoder	从左到右的单向注意力	训练效率高，生成任务效果优异，zero-shot能力强	理解抽取任务上相比encoder-decoder模式无优势，无法利用双向上下文	文本生成、对话系统、代码生成等
Prefix Decoder	在Decoder前加入“Prefix”，即额外的编码器生成的上下文信息作为条件输入	兼顾理解和生成，灵活性强	效率不如Causal Decoder，理解能力有限	条件生成、提示词补全，system prompt adaptation等



训练目标: Causal Decoder vs. prefix decoder

$$\mathcal{L}_{LM}(x) = \sum_{i=1}^n \log P(x_i | x_{<i})$$

根据已给出的上下文序列 ($x_{<i}$), 预测序列中下一个词 (x_i)。

$$\mathcal{L}_{DAE}(x) = \log P(x | \tilde{x})$$

$\tilde{x}$ 是经过扰乱的输入序列。

•Causal Decoder:

- 训练效率较高, 因为它的训练目标是在整个输入序列 (所有 token) 上逐步计算损失。
- 自然适合自回归式生成任务, 如文本生成、对话生成。
- 长文本生成
- 纯生成任务

•Prefix Decoder:

- 训练效率较低, 因为其只在输出 token (目标生成部分) 上计算损失。
- 需要处理前缀部分与生成内容之间的融合 (通过自注意力机制), 计算开销更大。
- 条件生成
- 少样本/零样本学习



tokenizer

男儿何不带吴钩，收取关山五十州。

模型	词表大小	分词结果	长度
LLaMA	32000	['男', '<0xE5>', '<0x84>', '<0xBF>', '何', '不', '<0xE5>', '<0xB8>', '<0xA6>', '<0xE5>', '<0x90>', '<0xB4>', '<0xE9>', '<0x92>', '<0xA9>', ',', ',', '收', '取', '关', '山', '五', '十', '州', '。']	24
Chinese LLaMA	49953	['男', '儿', '何', '不', '带', '吴', '钩', ',', ',', '收取', '关', '山', '五十', '州', '。']	14
ChatGLM-6B	130528	['男儿', '何不', '带', '吴', '钩', ',', ',', '收取', '关山', '五十', '州', '。']	11
ChatGLM2-6B	65024	['男', '儿', '何', '不', '带', '吴', '钩', ',', ',', '收取', '关', '山', '五十', '州', '。']	14
Bloom	250880	['男', '儿', '何不', '带', '吴', '钩', ',', ',', '收取', '关', '山', '五十', '州', '。']	13
Falcon	65024	['男', '儿', '❖', '❖', '❖', '不', '带', '吴', '❖', '❖', '❖', '❖', '❖', '收', '取', '❖', '❖', '山', '五', '十', '州', '。']	22

Qwen-7b 151643 13

Inf-34b 96536 12



tokenizer

GPT-4o & GPT-4o mini

GPT-3.5 & GPT-4

GPT-3 (Legacy)

<https://platform.openai.com/tokenizer>

如何运用大语言模型来辅助科学研究?

Clear

Show example

Tokens

22

Characters

17

如何运用大语言模型来科学?

Text

Token IDs



tokenization

大语言模型的分词器 (tokenizer) 可以用数学函数来描述, 它将输入字符串映射到一系列令牌 (tokens)。具体来说, 分词器可以被定义为一个函数:

$$T: \mathcal{S} \rightarrow \mathcal{V}^*$$

其中:

- $\mathcal{S}$ 是所有可能的输入字符串的集合。
- $\mathcal{V}$ 是预定义的有限令牌词汇表。
- $\mathcal{V}^*$ 是词汇表中令牌的所有可能序列。

对于任意输入字符串 $s \in \mathcal{S}$, 分词器函数 T 将其映射为一个令牌序列:

$$T(s) = [t_1, t_2, \dots, t_n]$$

满足以下条件:

1. 每个令牌 $t_i \in \mathcal{V}$ 。
2. 这些令牌的连接等于原始字符串, 即:

$$s = t_1 \cdot t_2 \cdot \dots \cdot t_n$$

其中 “ $\cdot$ ” 表示字符串连接操作。

合并的原则

1. 基于频率的合并

合并的核心原则是基于符号对在训练语料中的出现频率。具体步骤如下:

- 统计相邻符号对的频率: 对于当前的符号序列集合 S , 计算所有可能的相邻符号对 (a, b) 的出现次数 $f(a, b)$ 。

$$f(a, b) = \text{Count of occurrences of } (a, b) \text{ in } S$$

- 选择最高频的符号对: 找到出现次数最多的符号对 (a^*, b^*) :

$$(a^*, b^*) = \arg \max_{(a, b)} f(a, b)$$

- 合并符号对: 将 (a^*, b^*) 合并为新的符号 c , 并更新符号序列:

$$S' = S \text{ with all occurrences of } (a^*, b^*) \text{ replaced by } c$$

2. 更新频率

在每次合并后, 需要重新计算相邻符号对的频率, 因为符号序列已发生变化。



位置编码

训练式位置编码 (Learned Positional Encoding)

•**原理**：将位置编码当作可训练参数，通过训练模型时优化得到一个固定的「位置编码矩阵」。

•**特点**：

- GPT-3 等模型采用了这种方式。
- **优点**：是训练得到的位置编码可以适应具体的任务。
- **缺点**：这种方法没有「外推性」(Extrapolation)，也就是说，当输入序列长度超过训练时的最大长度时，位置编码无法很好地处理这种情况。

公式表示：对于序列长度为 n ，隐藏维度为 d 的输入：

$$\text{Input With Position Encoding} = \text{Token Embedding} + \text{Positional Encoding}$$

其中：

- Token Embedding $\in \mathbb{R}^{n \times d}$ ：输入的 token 嵌入矩阵。
- Positional Encoding $\in \mathbb{R}^{n \times d}$ ：可训练的位置嵌入矩阵。

位置编码在训练过程中更新，与 token embedding 一起参与优化。

旋转位置编码 (Rotary Positional Embedding, RoPE)

•**原理**：这种编码方法在每一层 Transformer 的 Self-Attention 模块中对 Q（查询向量）和 K（键向量）进行位置编码后，再计算 Attention Score。

•**特点**：

- RoPE 基于绝对位置编码实现了相对位置编码。
- 具有良好的外推性，因此即使输入序列长度超过训练时的范围，模型仍能较好地处理。
- 被许多语言模型采用，如 LLaMA、GLM-130B、PaLM。

•**优点**：

- 在保留绝对位置信息的基础上，可以自然地实现相对位置建模。
- 通过旋转操作高效实现。

$$\begin{aligned} Q_{\text{rot}} &= Q \cdot R \\ K_{\text{rot}} &= K \cdot R \\ \text{Attention Score} &= Q_{\text{rot}} \cdot K_{\text{rot}}^T \end{aligned} \quad R = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

ALiBi (Attention with Linear Biases)

•**原理**：在计算 Attention Score（注意力得分）后，直接为注意力得分矩阵增加一个预设的「偏置矩阵」。

•**特点**：

- ALiBi 位置编码设计简单。
- 它的偏置矩阵使模型具有良好的外推性，能够更好地处理超长序列。
- BLOOM 模型使用了这种位置编码。

•**优点**：

- 通过简单的偏置调整，使得注意力机制可以很好地处理序列长度扩展问题。
- 实现成本低，计算效率高。

$$\begin{aligned} \text{Attention Score}_{ij} &= Q_i \cdot K_j^T + b_{ij} \\ b_{ij} &= -|i - j| \cdot m \end{aligned}$$



Layer norm

Post-LN (Layer Normalization 在残差连接之后)

- 结构特点：
 - Layer Normalization 放在残差连接的后面，即对「残差计算的结果」进行归一化处理。
 - 公式：
$$x_{l+1} = \text{LayerNorm}(x_l + \text{SubLayer}(x_l))$$
其中 $\text{SubLayer}(x_l)$ 可以是 Attention 或 FFN。
- 优点：
 - 残差连接的结果经过 Layer Normalization，直接稳定了后续层的输入分布。
- 缺点：
 - 训练不稳定：
 - 在深层模型中（如几十层或更深的 Transformer），梯度容易因为归一化的顺序而变得不稳定，尤其是在早期训练阶段。
 - 导致梯度难以向残差路径传播，可能出现梯度爆炸或消失问题。
- 适用场景：
 - Transformer 的早期版本（如原始的 Transformer 架构）使用了 Post-LN。

Pre-LN (Layer Normalization 在残差连接中)

- 结构特点：
 - Layer Normalization 放在残差连接的前面，即对「输入」先进行归一化，再进行子层的计算。
 - 公式：
$$x_{l+1} = x_l + \text{SubLayer}(\text{LayerNorm}(x_l))$$
- 优点：
 - 训练更加稳定：
 - Pre-LN 在子层计算之前对输入进行归一化，确保了子层的输入分布稳定。
 - 梯度能够更有效地向前传播，减轻了梯度消失或爆炸的问题。
 - 更适合深层 Transformer（如 GPT-3、BERT 等大模型）。
- 缺点：
 - 模型效果略差：
 - 与 Post-LN 相比，在某些任务中模型的最终性能可能稍有下降（但这种差距通常很小）。
 - 主要是因为残差路径的分布受到 LayerNorm 的干扰，可能对模型学习能力有轻微影响。
- 适用场景：
 - 现代 Transformer 架构（如 GPT-3、BERT）大多使用 Pre-LN，以增强训练稳定性。

Sandwich-LN

- 结构特点：
 - 在 Pre-LN 的基础上额外插入一个 Layer Normalization。即：
 - 在子层的输入之前进行一次 Layer Normalization。
 - 在子层的输出后再进行一次 Layer Normalization。
 - 公式：
$$x' = \text{LayerNorm}(x_l), \quad x'' = \text{SubLayer}(x'), \quad x_{l+1} = \text{LayerNorm}(x_l + x'')$$
- 优点：
 - 梯度更加稳定：
 - 双层 Layer Normalization 能有效缓解梯度爆炸问题（如 Cogview 等模型中用于长序列生成任务）。
 - 适合对数值不稳定的任务，尤其是生成式任务。
- 缺点：
 - 训练可能崩溃：
 - 如果参数初始化不当或学习率较大，可能因为多次归一化导致训练不收敛。
 - 训练效率较低：额外的 Layer Normalization 增加了计算开销。
- 适用场景：
 - 特别复杂的生成任务（如长文本生成、大型模型训练）。
 - 需要非常精细地控制数值稳定性时使用。

Layer norm 的位置

1. Post LN: layer norm在残差链接之后

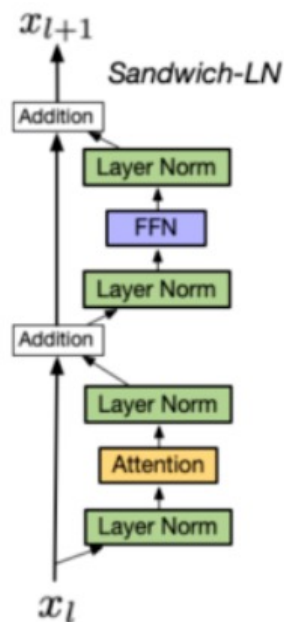
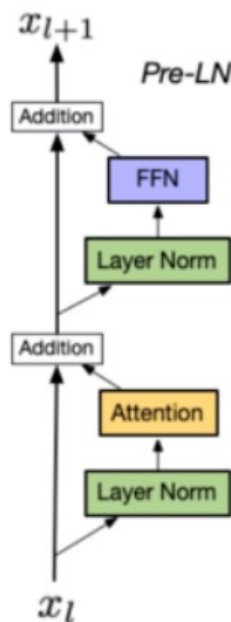
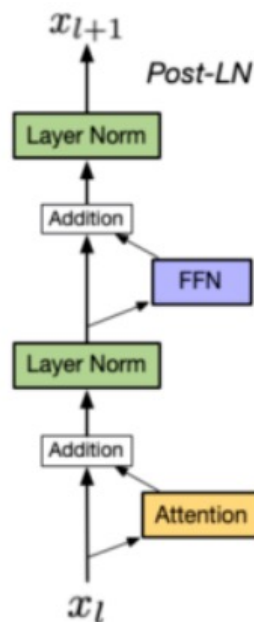
- 使用post-LN的深层transformer容易出现训练不稳定的问题。

2. Pre-LN: layer norm在残差链接中

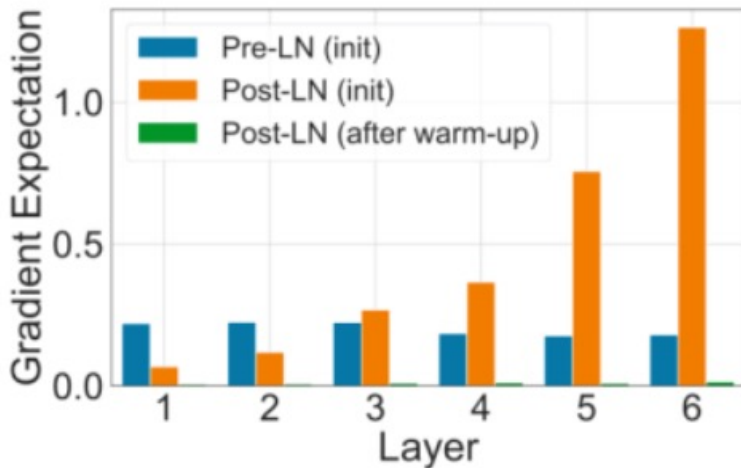
- 相比于Post-LN, 使用Pre-LN的深层transformer训练更稳定, 可以缓解训练不稳定问题。
- 相比于Post-LN, Pre-LN的模型效果略差。

3. Sandwich-LN: 在pre-LN的基础上, 额外插入了一个layer norm

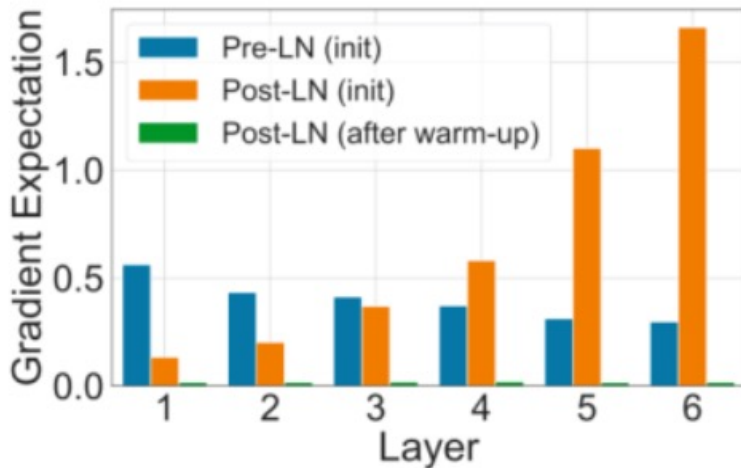
- Cogview用来避免值爆炸的问题。
- 缺点: 训练不稳定, 可能会导致训练崩溃。



Layer norm 的位置



(a) W^1 in the FFN sub-layers

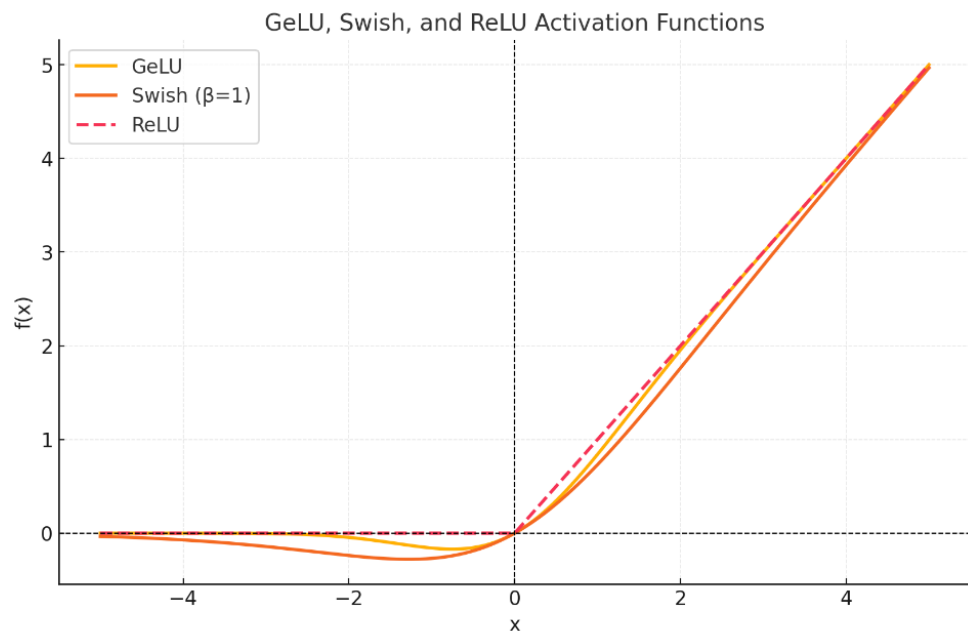


(b) W^2 in the FFN sub-layers

Post-LN在深层的梯度范数逐渐增大，导致了训练的不稳定性；
Pre-LN在每层的梯度范数近似相等，有利于训练稳定性。

模型	normalization
GPT3	Pre layer Norm
LLaMA	Pre RMS Norm
baichuan	Pre RMS Norm
ChatGLM-6B	Post Deep Norm
ChatGLM2-6B	Post RMS Norm
Bloom	Pre layer Norm
Falcon	Pre layer Norm

GeLU, Swish, And ReLU Activation Functions



FFN块的计算公式: $FFN(x) = f(xW_1 + b_1)W_2 + b_2$

GeLU激活函数: $GeLU(x) \approx 0.5x(1 + \tanh(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)))$

Swish激活函数: $Swish_{\beta}(x) = x \cdot \sigma(\beta x)$

2个可训练权重矩阵, 中间维度为4h

使用GLU线性门控单元的FFN块: $GLU(x) = \sigma(xW + b) \otimes xV$
 $FFN_{GLU} = (f(xW_1) \otimes xV)W_2$

使用GeLU的GLU: $GeGLU(x) = GeLU(xW) \otimes xV$

使用Swish的GLU: $SwiGLU = Swish_{\beta}(xW) \otimes xV$

3个可训练权重矩阵, 中间维度为4h * 2 / 3



激活函数

模型	激活函数
GPT3	GeLU
LLaMA	SwiGLU
LLaMA2	SwiGLU
baichuan	SwiGLU
ChatGLM-6B	GeLU
ChatGLM2-6B	SwiGLU
Bloom	GeLU
Falcon	GeLU

LLaMA
-7B

Modules	params_shape	params_num
model.embed_tokens.weight	[32000, 4096]	131072000
model.layers.0.self_attn.q_proj.weight	[4096, 4096]	16777216
model.layers.0.self_attn.k_proj.weight	[4096, 4096]	16777216
model.layers.0.self_attn.v_proj.weight	[4096, 4096]	16777216
model.layers.0.self_attn.o_proj.weight	[4096, 4096]	16777216
model.layers.0.mlp.gate_proj.weight	[11008, 4096]	45088768
model.layers.0.mlp.down_proj.weight	[4096, 11008]	45088768
model.layers.0.mlp.up_proj.weight	[11008, 4096]	45088768
model.layers.0.input_layernorm.weight	[4096]	4096
model.layers.0.post_attention_layernorm.weight	[4096]	4096

BLOOM
-7B

Modules	params_shape	params_num
transformer.word_embeddings.weight	[250880, 4096]	1027604480
transformer.word_embeddings_layernorm.weight	[4096]	4096
transformer.word_embeddings_layernorm.bias	[4096]	4096
transformer.h.0.input_layernorm.weight	[4096]	4096
transformer.h.0.input_layernorm.bias	[4096]	4096
transformer.h.0.self_attention.query_key_value.weight	[12288, 4096]	50331648
transformer.h.0.self_attention.query_key_value.bias	[12288]	12288
transformer.h.0.self_attention.dense.weight	[4096, 4096]	16777216
transformer.h.0.self_attention.dense.bias	[4096]	4096
transformer.h.0.post_attention_layernorm.weight	[4096]	4096
transformer.h.0.post_attention_layernorm.bias	[4096]	4096
transformer.h.0.mlp.dense_h_to_4h.weight	[16384, 4096]	67108864
transformer.h.0.mlp.dense_h_to_4h.bias	[16384]	16384
transformer.h.0.mlp.dense_4h_to_h.weight	[4096, 16384]	67108864
transformer.h.0.mlp.dense_4h_to_h.bias	[4096]	4096

$$4h = 4 * 4096 = 16384$$

$$\frac{2}{3} * 4h = 10922 \rightarrow 11008$$

$$11008/128 = 86$$



Layer norm 的方法

Layer Norm

$$\mu = E(X) \leftarrow \frac{1}{H} \sum_{i=1}^H x_i$$
$$\sigma \leftarrow \text{Var}(x) = \sqrt{\frac{1}{H} \sum_{i=1}^H (x_i - \mu)^2 + \epsilon}$$
$$y = \frac{x - E(x)}{\sqrt{\text{Var}(X) + \epsilon}} \cdot \gamma + \beta$$

gamma: 可训练的再缩放参数
beta: 可训练的再偏移参数

RMS Norm (均方根Norm)

$$\text{RMS}(x) = \sqrt{\frac{1}{H} \sum_{i=1}^H x_i^2}$$
$$x = \frac{x}{\text{RMS}(x)} \cdot \gamma$$

RMS Norm简化了Layer Norm, **去除掉计算均值进行平移**的部分。

对比LN, RMS Norm的**计算速度更快**。效果基本相当, 甚至略有提升。

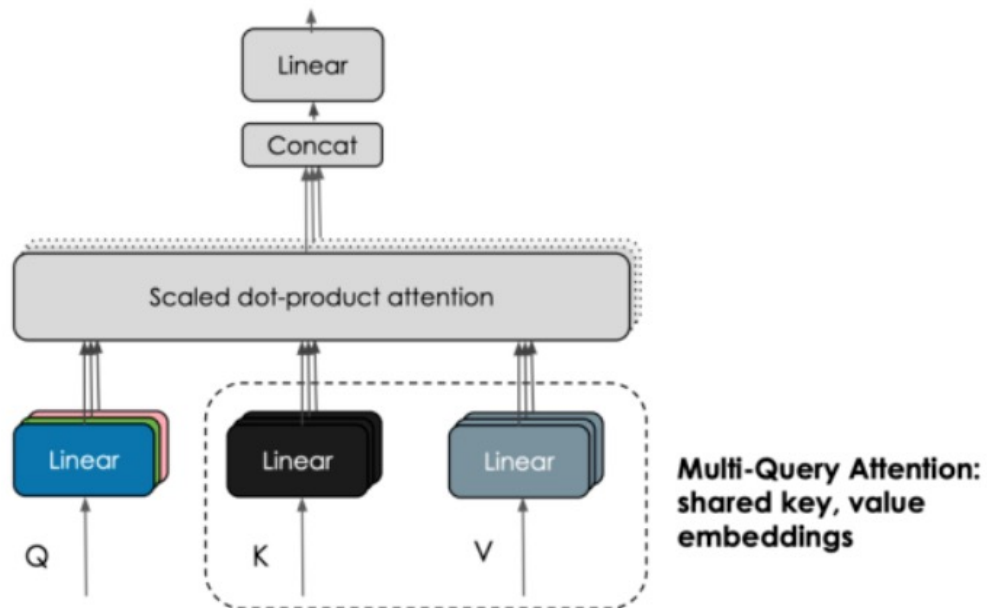
Deep Norm

```
def deepnorm(x):  
    return LayerNorm(x *  $\alpha$  + f(x))  
  
def deepnorm_init(w):  
    if w is ['ffn', 'v_proj', 'out_proj']:  
        nn.init.xavier_normal_(w, gain= $\beta$ )  
    elif w is ['q_proj', 'k_proj']:  
        nn.init.xavier_normal_(w, gain=1)
```

Deep Norm可以缓解爆炸式模型更新的问题, 把模型更新限制在常数, 使得模型训练过程更稳定。

Deep Norm方法在执行Layer Norm之前, up-scale了残差连接 ($\alpha > 1$); 另外, 在初始化阶段down-scale了模型参数($\beta < 1$)。

Multi Query attention



原理比较:

Multi-head Attention: 每个注意力头都有各自的query、key和value。

Multi-query Attention: 在所有的注意力头上**共享key和value**。

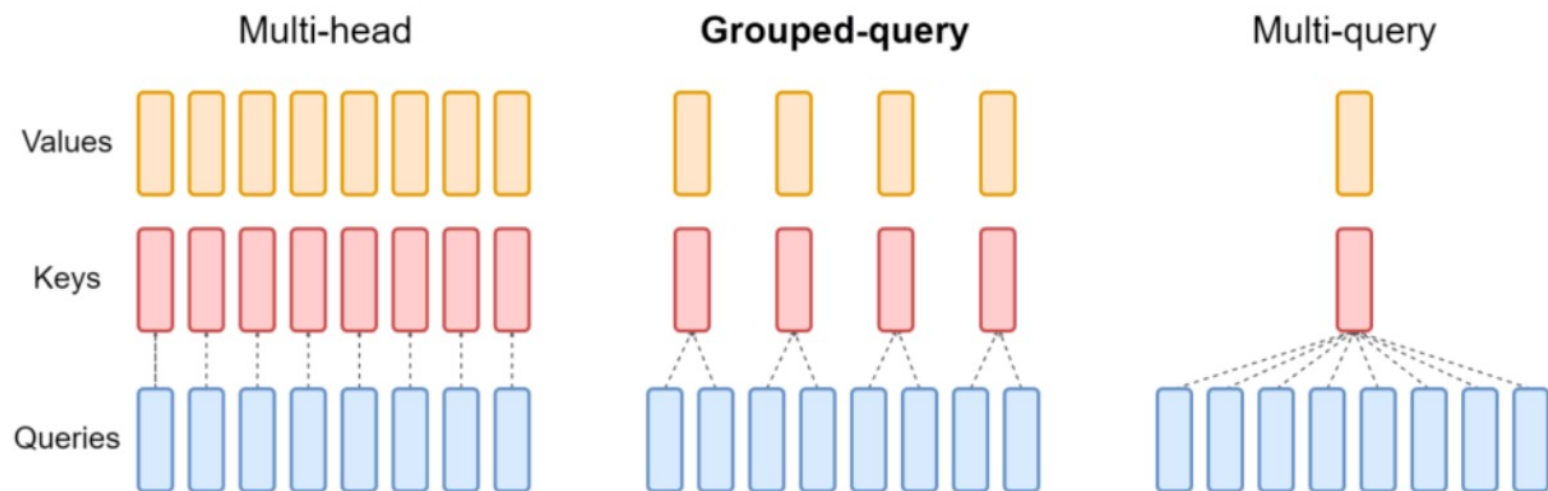
Falcon、PaLM使用了multi query attention。

动机与效果:

- 训练过程, 不会显著影响训练过程, 训练速度不变, 会引起非常细微的模型效果损失。
- 推断过程, **推理速度更快, 显存占用更低**。
 - 推理时, 反复地加载巨大的KV cache, 内存开销大, 性能是内存受限的。
 - 减少了KV cache的大小, 减少了显存占用, 提升了推理速度。



Group Query Attention



Grouped query attention: 介于multi head和multi query之间, 多个key和value。
ChatGLM2, LLaMA2-34B/70B使用了Grouped query attention。